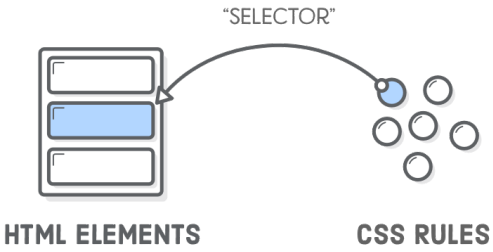
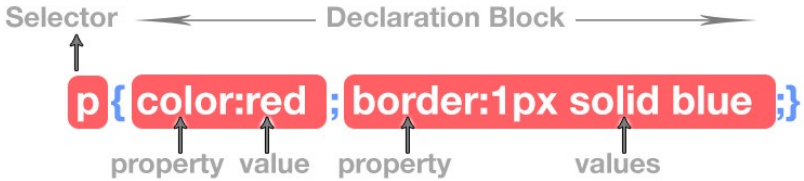
CSS - Cascading Style Sheets

Web-Design und Programmierung

What is CSS

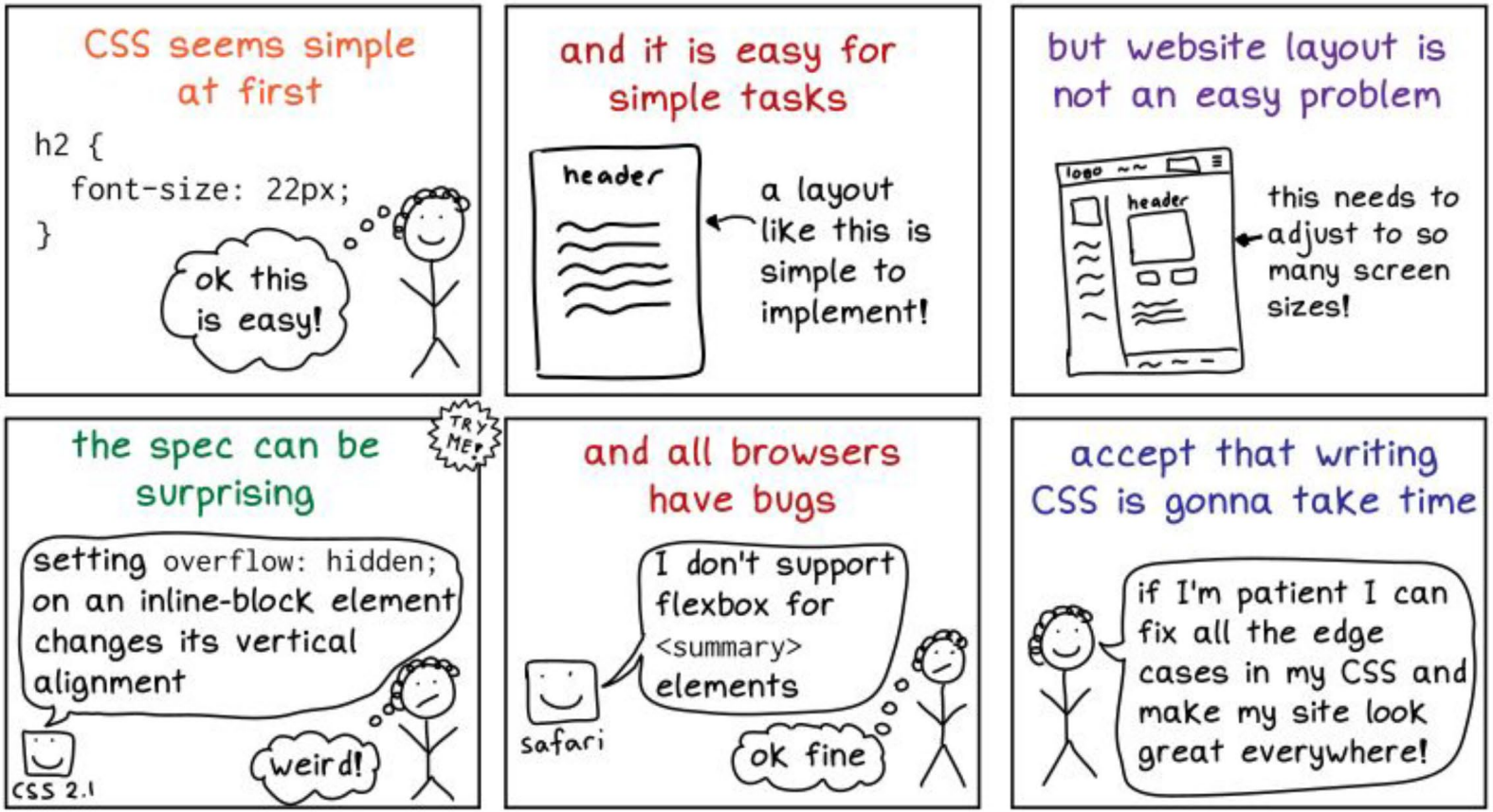
- CSS defines how your HTML document is presented to the user, i.e., how elements should be rendered on screen
  - It's all about layout, colors, fonts, sizes, etc…
- CSS is used to separate styling from content
- CSS consists of rules that define how elements of your HTML document look like
  - A rule **selects** an element and applies a set of **properties** to it.





JULIA EVANS
@b0rk

CSS isn't easy



Source: https://twitter.com/b0rk

Including Styles in HTML

- **Inline CSS**
  - …
  - Easy but hardy maintainable
  - No means to style “all” elements of a certain type
  - Try to avoid inline styles!
- **Internal CSS**
  - CSS code between <style> tags in the head of HTML document.
  - <style> /* CSS goes here */ </style>
  - High effort in case of multiple files
- **External CSS**
  - Link to the CSS file in the head of your HTML document
  - File ending .css
  - <link rel="stylesheet" href="styles.css"/>
  - Preferred variant
    - Flexible and reusable across several html files
    - Bandwidth savings

Inline CSS

```
<p style="color: blue;">This is a paragraph.</p>
```

Internal CSS

```
<head>
  <style type = text/css>
    body {background-color: blue;}
    p { color: yellow;}
  </style>
</head>
```

External CSS

```
<head>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
```

CSS- Comment

```
/* This syntax should be common to you*/
```

CSS Validation

- As for HTML, there are also validators for CSS
- Use is recommended
  - Easier debugging and troubleshooting
  - avoidance of non-obvious problems

- https://jigsaw.w3.org/css-validator/

- CSS Reference
  - https://developer.mozilla.org/en-US/docs/Web/CSS

Selectors

Give me the elements I need

- Basic Selectors
- Combined Selectors

Type Selector

- Applies style to all elements of a certain type, i.e., **HTML tag**

```
h1 {
        color: blue;
        font-weight: bold;
 }

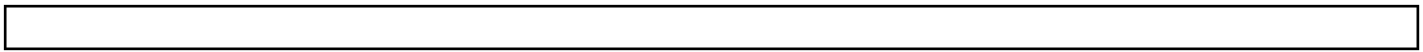
 <h1>a styled heading</h1>
```

ID-Selector

- Styling of an HTML Element with a specific, unique ID

```
#boldGreen {
        color: green;
        font-weight: bold;
 }

 <h1 id="boldGreen">a styled heading</h1>
```



Class Selector

- Class-Attribute can be used to “categorize” HTML Elements
- id vs class
  - **id** attribute can be used to uniquely identify elements -> identifier can only be used once per document.
  - **class** is not a unique identifier, but rather an assignment to a group
  - It is also possible for an element to belong to several classes, e.g. class=“important current”

```
.boldGreen {
    color: green;
    font-weight: bold;
}
<h1 class="boldGreen">a styled heading</h1>
<p class="boldGreen">a styled paragraph</p>
```

Class Selector

- Selects elements that contain **at least** the classes stated (e.g., both classes must be present in the example)

```
.bold.green {
    color: blue;
    font-weight: bold;
}

<h1 class="bold green">a styled heading</h1>
<p class="bold">an unstyled paragraph</p>
```

Universal Selector

- Selects all elements of an HTML document
- Useful for overriding browser default

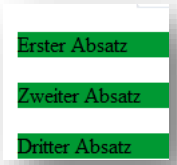
```
* {
      padding: 0px; margin: 0px;
  }
```

Basic Selectors - Summary

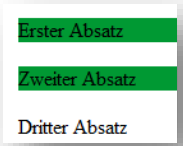
```
<body>
  <div id="box">
    <p class="one">Erster Absatz</p>
    <p class="one two">Zweiter Absatz</p>
    <p>Dritter Absatz</p>
  </div>
</body>
```

	Selector	Type	Description
	*	all	Selects all elements type-independently
1	E	Type	Selects all elements with the identifier E
2	.class	Class	Selects all elements with the named CSS class
3	.class1.class2	Class	Selects elements that contain at least the classes
4	#id	ID-selector	Selects an element with the passed ID

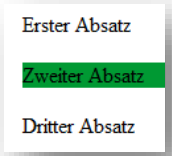
1
```
p {
  background-color: green;
}
```



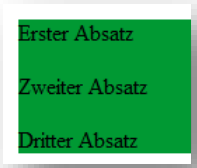
2
```
.one {
  background-color: green;
}
```



3
```
.one.two {
  background-color: green;
}
```



4
```
#box {
  background-color: green;
}
```



Group and Context Selectors

Selector	Description
sel1, sel2 … sel n	Selects the combination of all nodes selected by the selectors
E F	Selects all elements F that are **descendants** (not only children) of an element E
E > F	Selects all elements F that are **direct** child nodes of an element E
E + F	Selects **immediate next sibling** nodes of an element of type E from type F (muss direkt nachher kommen)
E ~ F	Selects **all the following sibling** nodes of an element E that have the type F

1 2 3 4 5

```
<div id="box">
  <p class="one">Erster Absatz</p>
  <p class="one two">Zweiter Absatz</p>
  <div>
    <p>Dritter Absatz</p>
    <p>Vierter Absatz</p>
  </div>
</div>
<p id="p5">Fuenfter Absatz</p>
<p id="p6">Sechster Absatz</p>
```

```
#box, #p5 {
    background-color: green;
}
```

1

Erster Absatz
Zweiter Absatz
Dritter Absatz
Vierter Absatz
Fuenfter Absatz
Sechster Absatz

```
#box p {
    background-color: green;
}
```

2

Erster Absatz
Zweiter Absatz
Dritter Absatz
Vierter Absatz
Fuenfter Absatz
Sechster Absatz

```
#box > p {
    background-color: green;
}
```

3

Erster Absatz
Zweiter Absatz
Dritter Absatz
Vierter Absatz
Fuenfter Absatz
Sechster Absatz

4

```
#box + p {
    background-color: green;
}
```

Erster Absatz
Zweiter Absatz
Dritter Absatz
Vierter Absatz
Fuenfter Absatz
Sechster Absatz

5

```
#box ~ p {
    background-color: green;
}
```

Erster Absatz
Zweiter Absatz
Dritter Absatz
Vierter Absatz
Fuenfter Absatz
Sechster Absatz

Attribute Selectors

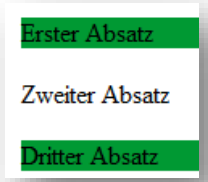
```
<body>
  <div id="box">
    <p title="P1">Erster Absatz</p>
    <p name="P2">Zweiter Absatz</p>
    <div title="div2">
     <p>Dritter Absatz</p>
    </div>
  </div>
</body>
```

	Selector	Description
1	[name]	Selects all elements of any type that have an attribute name
2	E[name]	Selects all elements of type E that have an attribute name
3	[name=value]	Selects all elements to which all named attribute filters apply
	[name|=value]	Especially for lang attributes; selects an element if the named attribute has the passed value, or the value followed by any hyphen-separated string.
	[name*=value]	Selects an element if the value of the named attribute contains the passed string as substring
	[name~=value]	Selects an element if the value of the named attribute contains the passed string as a partial value (separated by spaces)
	[name$=value]	Selects an element if the value of the attribute ends with the passed string
	[name!=value]	Selects an element if the value of the named attribute does not correspond to the passed string or the attribute does not exist
	[name^=value]	Selects an element if the value of the attribute starts with the passed string

(wissen dass es das gibt, kann man dann nachschauen)

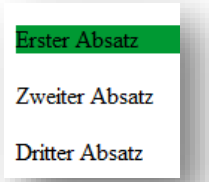
1

```
[title] {
  background-color: green;
}
```



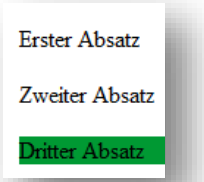
2

```
p[title] {
  background-color: green;
}
```



3

```
[title="div2"] {
  background-color: green;
}
```



Pseudo-Classes

- A CSS pseudo-class is a keyword added to a selector that specifies a special state of the selected element(s).
  - For example, the pseudo-class :hover can be used to select a button when a user's pointer hovers over the button and this selected button can then be styled.
- A pseudo-class consists of a colon (:) followed by the pseudo-class name (e.g., :hover).
  - A functional pseudo-class also contains a pair of parentheses to define the arguments.
  - The element that a pseudo-class is attached to is defined as an anchor element (e.g., button in case button:hover)
- See https://developer.mozilla.org/en-US/docs/Web/CSS/Pseudo-classes

Pseudo Classes

- Pseudo classes often used with links
  - **:link** -> Matches links that have not yet been visited
  - **:visited** -> Matches links that have been visited
  - **:hover** -> Matches when a user designates an item with a pointing device
  - **:active** -> Matches when an item is being activated by the user; for example, when the item is clicked on
  - **:focus** -> Matches when an element has focus

```
a:link { color: green; }
a:focus { background-color: orange;}
```

Tree-structural pseudo-classes

- **:empty** -> Represents an **element with no children** other than white-space characters.
- **:nth-child** -> Uses An+B notation to select **elements** from a **list of sibling elements**.
- **:nth-last-child** -> Uses An+B notation to select **elements** from a **list of sibling elements**, counting backwards from the end of the list.
- **:first-child** -> Matches an element that is the **first** of its **siblings**.
- **:last-child** -> Matches an element that is the **last** of its **siblings**.
- **:only-child** -> Matches an element that has no siblings.
- **:nth-of-type** -> Uses An+B notation to select **elements** from a **list of sibling elements** that match a **certain type** from a list of sibling elements.
- **:nth-last-of-type** -> Uses An+B notation to select **elements** from a **list of sibling elements** that match a certain type from a list of sibling elements counting backwards from the end of the list.
- **:first-of-type** -> Matches an element that is the first of its siblings, and also matches a certain type selector.
- **:last-of-type ->** Matches an element that is the last of its siblings, and also matches a certain type selector.
- **:only-of-type** -> Matches an element that has no siblings of the chosen type selector.

Tree-structural pseudo-classes

```
<body>
  <div id="box">
    <p>Erster Absatz</p>
    <p>Zweiter Absatz</p>
    <p>Dritter Absatz</p>
    <p>Vierter Absatz</p>
    <p>Fuenfter Absatz</p>
  </div>
  <div id="box2">
    <p>Erster Absatz</p>
    <p>Zweiter Absatz</p>
    <p>Dritter Absatz</p>
    <p>Vierter Absatz</p>
    <p>Fuenfter Absatz</p>
  </div>
  <div id="box3">
    <p>Erster Absatz</p>
  </div>
</body>
```

```
p:first-child {
  background-color: green;
}
```

Erster Absatz

Zweiter Absatz

Dritter Absatz

Vierter Absatz

Fuenfter Absatz

Erster Absatz

Zweiter Absatz

Dritter Absatz

Vierter Absatz

Fuenfter Absatz

Erster Absatz

```
p:only-child {
  background-color: green;
}
```

Erster Absatz

Zweiter Absatz

Dritter Absatz

Vierter Absatz

Fuenfter Absatz

Erster Absatz

Zweiter Absatz

Dritter Absatz

Vierter Absatz

Fuenfter Absatz

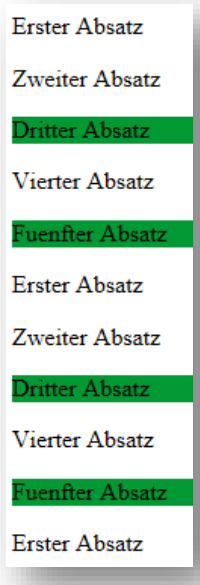
Erster Absatz

Tree-structural pseudo-classes

```
<body>
  <div id="box">
    <p>Erster Absatz</p>
    <p>Zweiter Absatz</p>
    <p>Dritter Absatz</p>
    <p>Vierter Absatz</p>
    <p>Fuenfter Absatz</p>
  </div>
  <div id="box2">
    <p>Erster Absatz</p>
    <p>Zweiter Absatz</p>
    <p>Dritter Absatz</p>
    <p>Vierter Absatz</p>
    <p>Fuenfter Absatz</p>
  </div>
  <div id="box3">
    <p>Erster Absatz</p>
  </div>
</body>
```

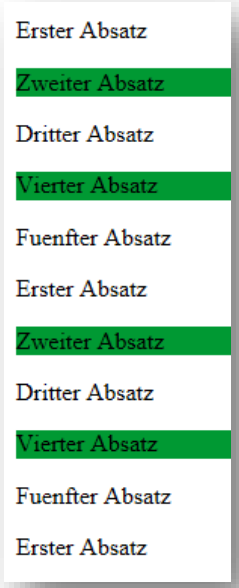
Every 2nd child element, if of type p, starting from the 3rd element

```
p:nth-child(2n+3) {
    background-color: green;
}
```



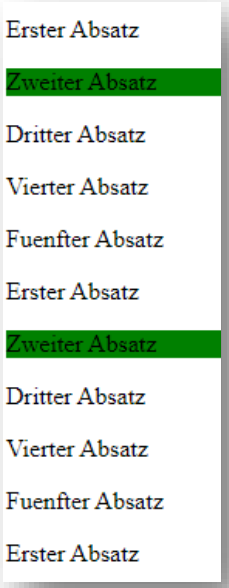
First child has index 1 and is therefore not selected

```
p:nth-child(even) {
    background-color: green;
}
```



Every 4th child element if of type p, starting from behind

```
p:nth-last-child(4n) {
    background-color: green;
}
```



Tree-structural pseudo-classes

```
<div id="box">
    <p>Erster Absatz</p>
    <p>Zweiter Absatz</p>
    <p>Dritter Absatz</p>
    <p>Vierter Absatz</p>
    <p>Fuenfter Absatz</p>
</div>
<div id="box2">
    <p>Erster Absatz</p>
    <p>Zweiter Absatz</p>
    <h2>Überschrift</h2>
    <p>Dritter Absatz</p>
    <p>Vierter Absatz</p>
    <p>Fuenfter Absatz</p>
</div>
<div id="box3">
    <p>Erster Absatz</p>
</div>
```

Every 2nd p child element

```
p:nth-of-type(2n) {
  background-color: green;
}
```

Erster Absatz

Zweiter Absatz

Dritter Absatz

Vierter Absatz

Fuenfter Absatz

Erster Absatz

Zweiter Absatz

Überschrift

Dritter Absatz

Vierter Absatz

Fuenfter Absatz

Erster Absatz

Every 2nd child element if of type p

```
p:nth-child(2n) {
  background-color: green;
}
```

Erster Absatz

Zweiter Absatz

Dritter Absatz

Vierter Absatz

Fuenfter Absatz

Erster Absatz

Zweiter Absatz

Überschrift

Dritter Absatz

Vierter Absatz

Fuenfter Absatz

Erster Absatz

CSS Properties

The basic stuff

- Font formatting
- Alignment
- Background
- List

Font formatting

- Deals with
  - Font size, Font type, …
- Typical properties
  - font-family: Family1, …, Family_n;
  - font-style: italic/normal/oblique
  - font-variant: normal/small-caps
  - font-size: Value (Value+Unit, xx-small - xx-large)
  - font-weight: lighter/normal/bold/bolder/…
  - font-stretch: normal/narrower/condensed/wider/…
  - word-spacing: Value
  - letter-spacing: Value
  - text-decoration: underline/overline/line-through/blink/none
  - text-transform: capitalize/lowercase/uppercase/normal
  - color: Color-value
- https://wiki.selfhtml.org/wiki/CSS/Wertetypen

Alignment

- For texts
  - Alignment: e.g. left- or right-aligned
    - In tables often also vertical alignment
    - Indentation
    - Line height
  - text-indent: text indentation
    - Value (positive moves to right, negative to left)
    - Normal only first line of a paragraph
  - line-height: Line height (Value)
  - vertical-align: Vertical alignment
    - Baseline, top, middle, bottom,…
    - See https://developer.mozilla.org/en-US/docs/Web/CSS/vertical-align
  - text-align: Horizontal alignment
    - start, end, center, justify, justify-all
  - white-space: Text wrap

Background-Color and -Images

- **background-color:** Color value
- **background-image:** Url to image
- **background-repeat:**
  - background image can be repeated along the horizontal and vertical axes
  - Values: repeat-x (horizontally only), repeat-y (vertically only), repeat (covers the whole background), space (repeated, but not clipped), round (repeated images will stretch, leaving no gaps, until there is room for another one to be added), no-repeat
- **background-attachment:**
  - Values:
    - scroll: background is fixed relative to the element itself and does not scroll with its contents
    - fixed: background is fixed relative to the viewport; even if an element has a scrolling mechanism, the background doesn't move with the element
    - local: background is fixed relative to the element's contents; if the element has a scrolling mechanism, the background scrolls with the element's contents
- **background-position:** top, bottom, left, right, center, 2 values (x and y)
- **background:** Shorthand (Combination of the above values)

List

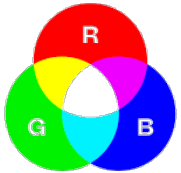
- For bullet list (<ul>) and enumerations (<ol>)
- **list-style-type:** Element shown
  - see https://developer.mozilla.org/en-US/docs/Web/CSS/list-style-type for possible values
- **list-style-position:**
  - position of the marker relative to a list item
    - Inside | outside (default)
- **list-style-image:** URL to own graphics used as marker
- **list-style:** shorthand property

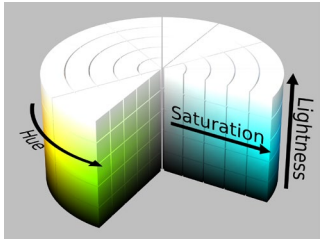
Colors, Fonts and Sizes

Make it nicer!

Color Values

- Color keywords
  - white, black, red, blue, darkslateblue, darkorange, firebrick,…
- Hexadecimal RGB values (#RRGGBB)
  - #FFFFFF, #000000, #FF0000, #0000FF, #483D8B, #FF8C00, #B22222,…
- RGB(A) values or HSL(A) values (hue, saturation, lightness)
  - rgb(255, 255, 255),
  - rgba(72, 61, 139, 0.5) -> alpha channel: 0-1 (transparent - opaque)
  - hsl(0, 100%, 50%) -> hue between 0 - 360





Units

- **Absolute Values**
  - px: pixel (absolute value)
- **Relative Values**
  - Depending on the size of e.g., parent element or font size
  - %: relative to parent's size
  - em: the current element's font size
    - (when used as unit for font-size, em refers to the parent-element's font size)
  - rem: relative to the root font size
    - (font-size of html element; default is 16px)
  - vw/vh: 1% of the viewport width/height

- More about CSS units: Values and units on MDN

Web Fonts

- The font-family property takes one or more font names (separated by a comma)
- Browser uses the first available font from the list (user might not have all fonts installed)
  - `font-family: Helvetica, Arial, sans-serif;`
- Include web fonts in your project if you don’t want to be limited to the fonts installed on your user’s computer
  - Where to find web fonts: Google Fonts, Font Squirrel, Fontspring,…
  - Web Open Font Format (.woff) is supported by all modern browsers (https://caniuse.com/?search=woff)

Web Fonts

- @font-face
- Declare a font family with a specific name
- Specify the source file
- Do not use links to (Google) fonts
  - Privacy/legal issues

```
@font-face{
   font-family: myFont;
   src: url(font.woff);
}
div{
   font-family: myFont, ..., serif/sans-serif;
}
```

Inheritance, Specifity And Cascading

Now it’s getting weird!

- Inheritance
- Specifity
- Cascading

Reusing Styles

```
<section>
  <h1>Headline</h1>
  <p>Some text</p>
</section>
```

```
h1 {
  color: #454545;
  font-family: Arial;
}

p {
  color: #454545;
  font-family: Arial;
}
```

Group selectors, separate them by comma

```
h1, p {
  color: #454545;
  font-family: Arial;
}
```

Elements **h1** and **p** inherit styles from their parent **section**

```
section {
  color: #454545;
  font-family: Arial;
}
```

CSS Inheritance

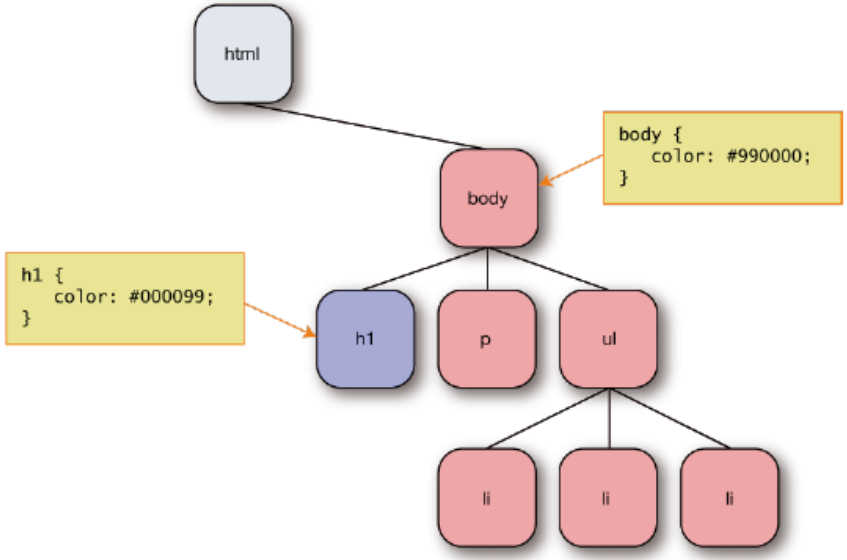
- There are two types of CSS properties:
  - inherited properties - are set to the **computed value of the parent** element - e.g. “color”
    - List of properties see https://web.dev/learn/css/inheritance
  - non-inherited properties - are set to the **initial value of the property** - e.g. “border”
- In the example below, the color and font-family values are inherited by the h1 and p elements from their section parent.

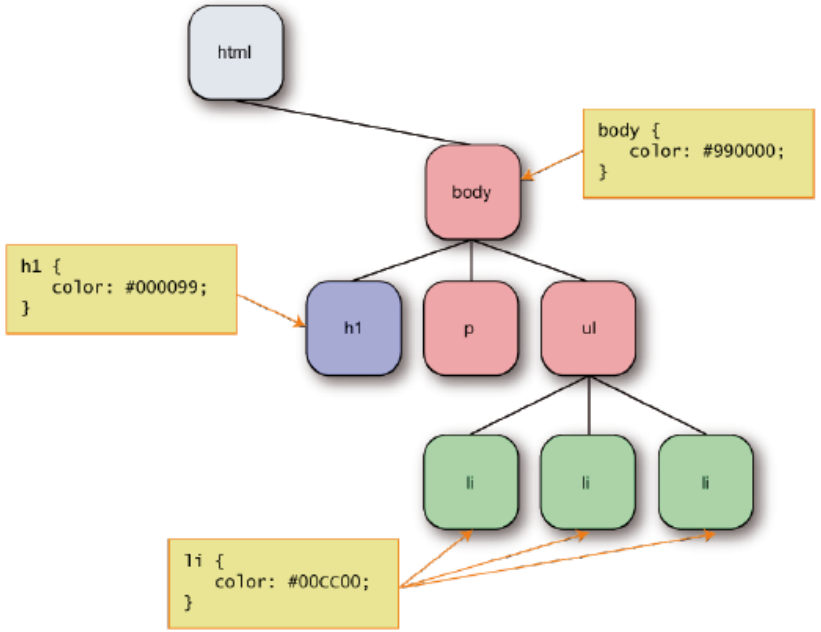
```
<section>
  <h1>Headline</h1>
  <p>Some text</p>
</section>
```

```
section {
  color: #454545;
  font-family: Arial;
}
```

CSS Inheritance

- Styles are inherited as long as specific elements do not redefine them





CSS Specificity

- Sometimes it is hard to know, which CSS rule is applied
- We can determine how specific a (combined) selector is by adding the weight of the individual selectors.
- Order of specificity (form general to specific)
  - Universal selector
  - Typselector
  - Classselector
  - Attributselector
  - Pseudoclasses
  - ID-Selector
  - Inline style

CSS Specificity

- Specificity is the algorithm used by browsers to determine which rule from competing CSS declarations gets applied to an element.
- specificity algorithm is basically a three-column value of three categories or weights
- ID column
  - For each ID in a matching selector, add 1-0-0 to the weight value.
- CLASS column
  - Includes class selectors, attribute selectors, and pseudo-classes. For each of them add 0-1-0 to the weight value.
- TYPE column
  - Includes type selectors, and pseudo-elements, and all other selectors with double-colon notation. For each of them, add 0-0-1 to the weight value

```
#introduction{ color: red;}
.message{ color: green;}
p{ color: blue;}
```

```
<p class="message" id="introduction">
    Beispieltext
</p>
```



Beispieltext

CSS Specificity

- We can calculate specificity by using a pattern similar to semantic versioning:
  - We use the following pattern: id.class.type
  - For example: 2.3.5 > 1.4.8
  - only the first unequal number matters
- Always try to avoid overly specific selectors!

```
section.contact form [type="text"] {…} ->
0.2.2

nav#primary a.active {…} ->
1.1.2
```



Specific rule

CSS Specificity

- The more specific your selector is, the more likely the rules get applied to the selected element.
- But sometimes other style declarations are really !important…
  - Important “overrides” specificity
- Try to avoid the important keyword!

```
header ul#navigation a {
  background-color: #b4e5f9;
  text-decoration: underline;
}
header a {
  text-decoration: none !important;
}
```

CSS Specificity

```
<p>This is my first paragraph!</p>

<p class="meta">
This is another paragraph.
</p>

<p class="meta" id="introduction">
This is the page introduction.
</p>

<p style="color: black;">
This is my last paragraph.
</p>
```

```
.meta {
  background-color: gray;
  color: lime;
}

p {
  color: white;
  background-color: rebeccapurple;
}

#introduction {
  background-color: red;
}
```

This is my first paragraph!

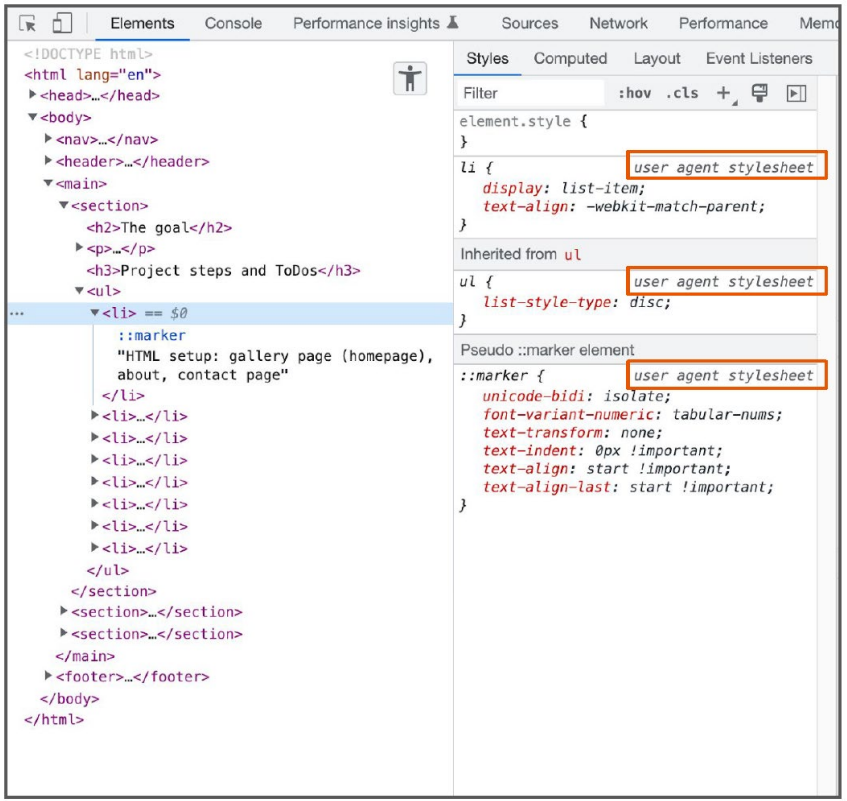
This is another paragraph.

This is the page introduction.

This is my last paragraph.

Where is the CSS code coming from?

- User agent stylesheets
  - styles defined by the browser
- Author stylesheets
  - what we developers build
- User stylesheets
  - configured in the browser by the user of your website



Cascading Style Sheets

different origins of
style sheets...

huge CSS files with
thousands of rules...

???

some styles are more
specific than others...

selectors occur more
often than once...

source order...

different types of
selectors...

inheritance...

Cascading Style Sheets

- If there's more than one property value that can be applied to an element… which one "wins"?
- Different CSS sources: browser styles, user style, external styles, page specific styles, inline styles
- CSS Specificity: elements, classes, IDs, inline styles
- CSS importance: !important
- CSS inheritance
- CSS transitions and animations
- And: source order matters! Rules that come later override earlier rules.

Cascading Style Sheets - Layer

- The **@layer** CSS rule declares a cascade layer and can also be used to define the order of precedence in case of multiple cascade layers
  - First layer is less specific

```
<p class="alert">Beware of the zombies</p>
```




```
@layer module, state;

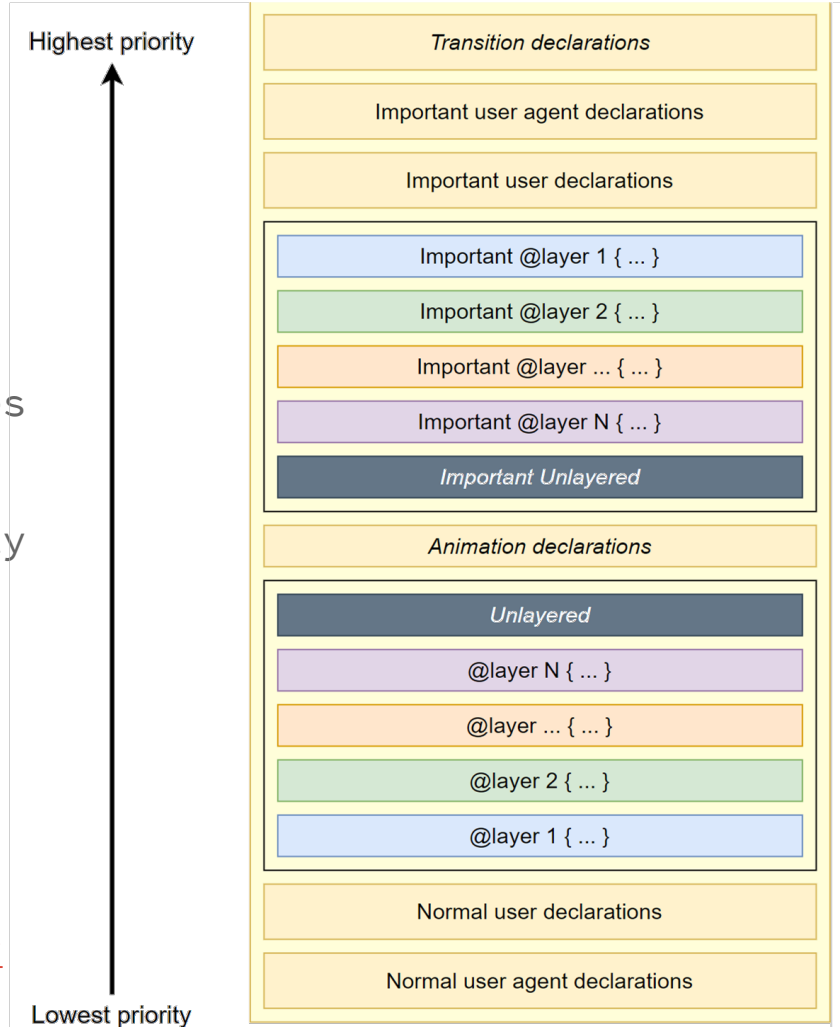
@layer state {
    .alert {
        background-color: red;
    }
    p {
        border: medium solid blue;
    }
}

@layer module {
    .alert {
        border: medium solid violet;
        background-color: yellow;
        color: white;
    }
}
```

Cascading Style Sheets

1. Relevance: get all the rules from all available sources that apply to a given element
2. Origin and importance: sort the rules by origin and importance
3. Specificity: when the origin is the same, rules are sorted by specificity
4. Order of appearance: when the specificity is the same, rules that can be found later in the code override earlier ones.

See also https://developer.mozilla.org/en-US/docs/Web/CSS/Cascade#cascading_order



Organisation of CSS Rules

- Define some base styles for your whole website or application using type selectors.
  - Set a font color, the font-size of text and headlines, list styles, link styles…
- Prefer class selectors over ID selectors. Why?
  - They are less specific than ID selectors or inline style, thus can be overridden easily when needed.
  - They can be combined: an element can have more than one class (but only one ID).
  - They can be reused.
- Sort rules from less to more specific.
- Try avoiding ID selectors, inline styles and !important.*
- Make use of cascade layers when overriding third-party styles.

CSS Box Model

Think inside the box!

Box Model

- On a website, every element is a “box”
- Block vs. inline boxes
  - Type refers to how the box behaves in terms of page flow and in relation to other boxes on the page.
  - Boxes have an **inner display type** and an **outer display type**.

- **block boxes**
  - , <p>, <h1>, <article>,…
- **inline boxes**
  - , <a>, <img>, <input>,…



Box Model – Outer Display Type Block

- If a box has an outer display type of **block**, then:
  - The box will break onto a **new line**.
  - The width and height properties are respected.
  - Padding, margin and border will cause other elements to be pushed away from the box.
  - If width is not specified, the box will extend in the inline direction to fill the space available in its container.
  - In most cases, the box will become as wide as its container, filling up 100% of the space available.
- Some HTML elements, such as <h1> and <p>, use block as their outer display type by default.

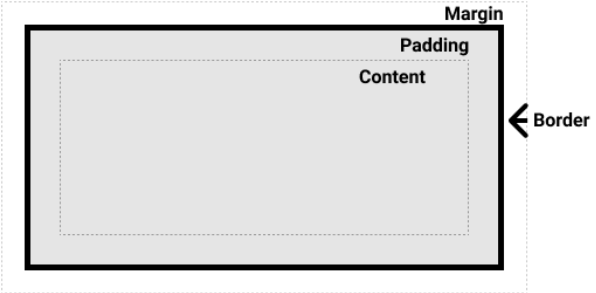
Box Model – Outer Display Type inline

- If a box has an outer display type of **inline**, then:
  - The box will not break onto a new line.
  - The width and height properties will not apply.
  - Top and bottom padding, margins, and borders will apply but will not cause other inline boxes to move away from the box.
  - Left and right padding, margins, and borders will apply and will cause other inline boxes to move away from the box.
- Some HTML elements, such as <a>, , and use inline as their outer display type by default.

Box Model – Inner Display Type

- Boxes also have an **inner display type**, which dictates how elements inside that box are laid out.
- Block and inline layout is the default way things behave on the web.
  - By default and without any other instruction, the elements inside a box are also laid out in normal flow and behave as block or inline boxes.
- **display** property sets an element's inner and outer display types (more on that when dealing with layouts)

Box Model



- **Content box:** The area where your content is displayed;
  - size it using properties like inline-size and block-size or width and height.
- **Padding box:** The padding sits around the content as white space;
  - size it using padding and related properties.
- **Border box:** The border box wraps the content and any padding;
  - size it using border and related properties.
- **Margin box:** The margin is the outermost layer, wrapping the content, padding, and border as whitespace between this box and other elements;
  - size it using margin and related properties.

Box Model - Examples

- Use measurements for padding and margin
  - `.box { margin: 12px; }`
  - ```
    .box {
      padding-top: 32px;
      padding-bottom: 12px;
    }
    ```
- Define a border width, style and color
  - `.box { border: 2px dashed violet; }`
    ```

Box Model – Shorthand Properties

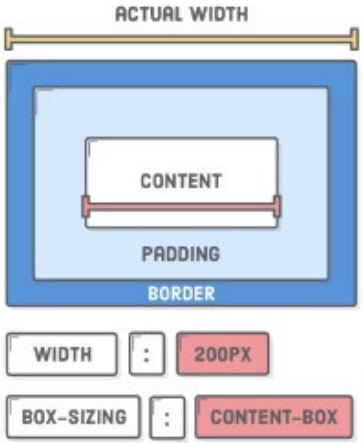
- Let you set values of multiple CSS properties simultaneously.
- Can be used for: border, margin, padding, background, font, …

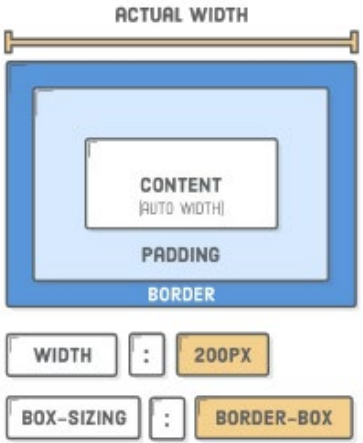
```
border-width: 2px;
border-style: solid;
border-color: red;
border: 2px solid red;
```

```
margin-top: 5px;
margin-bottom: 15px;
margin-right: 10px;
margin-left: 10px;
margin: 5px 10px 15px 10px;
```

Box Sizing

- Width and height define dimensions of a box's content.
  - Paddings and borders are additionally added.
- To define width and height of an object including paddings and borders, set the **box-sizing** property from content-box (default) to border-box.

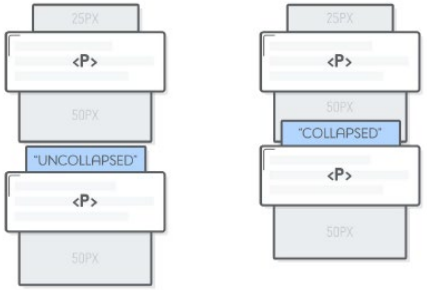




Vertical Collapsing of Marging

- Check the following example!
  - Instead of sitting 48px apart, their 24px margins merge together, occupying the same space!

```
<style>
  p {
    margin-top: 24px;
    margin-bottom: 24px;
  }
</style>
<p>Paragraph One</p>
<p>Paragraph Two</p>
```

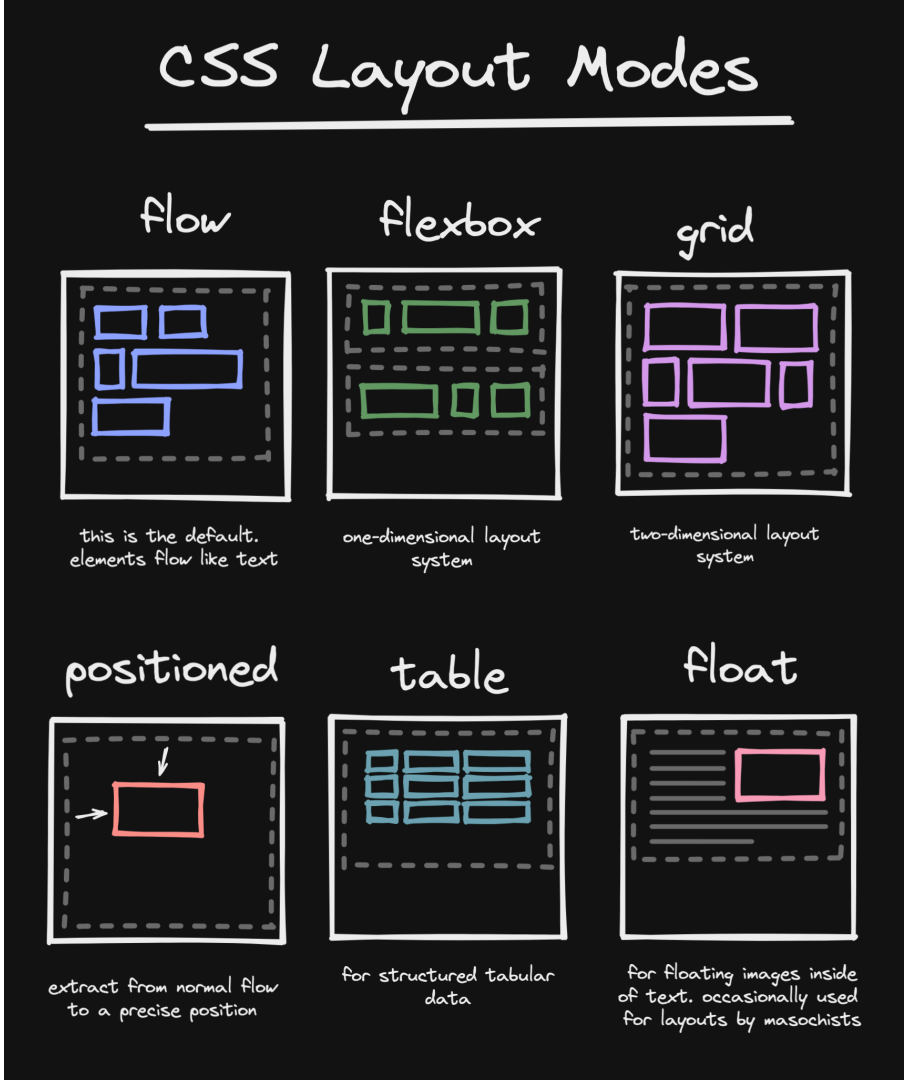


- **Only vertical** margins collapse (not horizontal)
- Only adjacent siblings collapse
  - E.g., if there would be a
 between the p's above, no collapsing would appear
- Bigger margin wins!
- For mor details check https://www.joshwcomeau.com/css/rules-of-margin-collapse/

Layouts in CSS

Layout algorithms in CSS

- **Flow Layout**
- Table Layout
- Float Layout
- **Positioned Layout**
- **Flexible box Layout (Flexbox)**
- **Grid Layout**

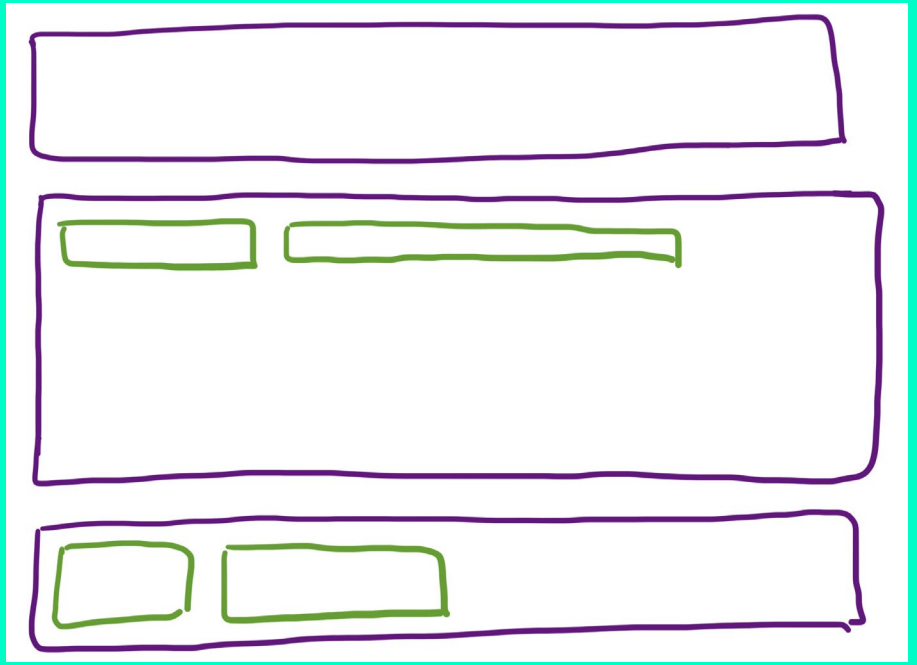


Display Property

- Sets whether an element is treated as block or inline element and the layout used for its child elements.
  - display-outside values: block, inline, inline-block, none (to hide elements)
  - display-inside values: flow, flex, grid

```
.element {
  display: block | inline | inline-block | none | flex | grid;
}
```

Flow Layout



Flow Layout (Default)

- **Block-level elements**
  - Always create a new line
  - Some default margin (top/bottom)
  - Uses all of the available space (left/right) dependent on the parent element
  - Examples: , <h*>, <p>, <ol>, <ul>, <table>
- **Inline elements**
  - Do not start a new line (also linebreaks may happen)
  - No default margin
  - Use only the space that is needed to show the content
  - Examples: <a>,
 (!), <img>, <input>, <select>, , <textarea>

```
<body>
    <div>Hallo</div><span>schöne</span>
    <span>neue</span><div>Welt</div>
</body>
```

Hallo
schöne neue
Welt

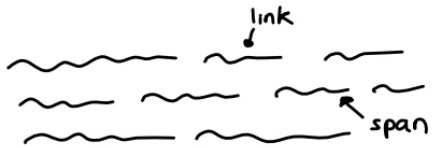
Julia Evans
@b0rk

inline vs block

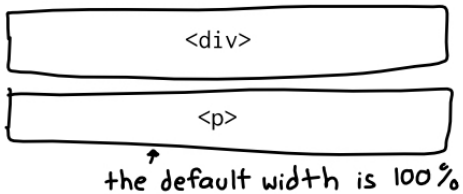
HTML elements default to inline or block

example block elements	example inline elements
<p>	<a>
	<i>
<h1> - <h6>	<button> <input>
<table> <form>	<small> <abbr>
<article> <nav>	<textarea>

inline elements are laid out horizontally



block elements are laid out vertically by default



inline elements ignore width & height

Setting the width is impossible, but you can use `line-height` to change the height

also, inline elements ignore the vertical padding of other inline elements

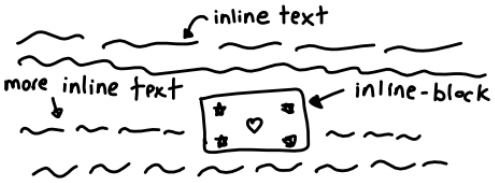
display can force an element to be inline or block

`display` determines 2 things:

1. whether the element itself is `inline`, `block`, `inline-block`, etc
2. how child elements are laid out (`grid`, `flex`, `table`, default, etc)

display: inline-block;

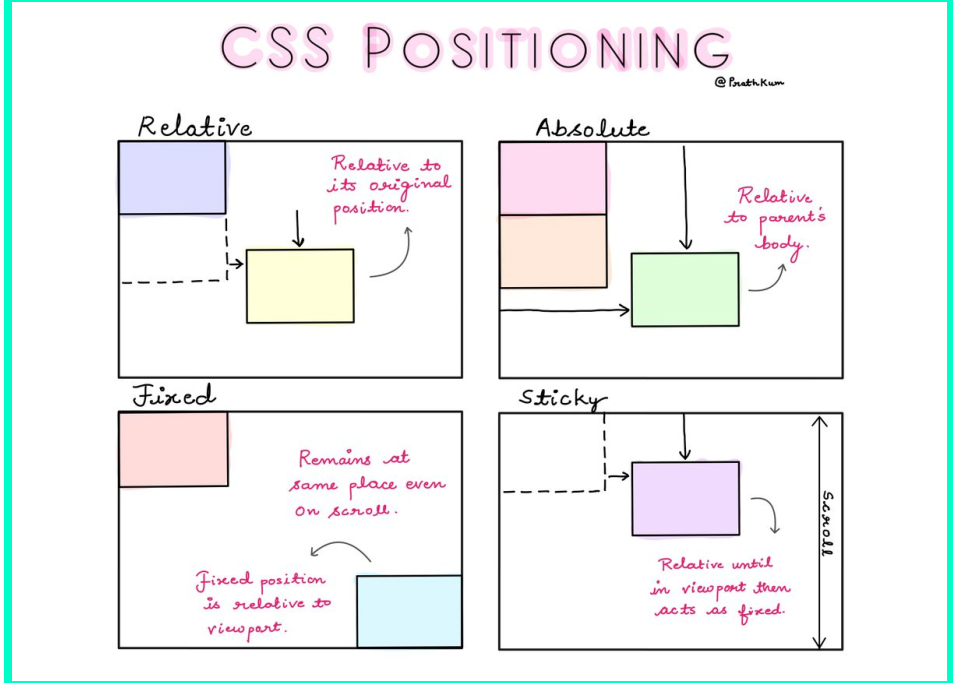
`inline-block` makes a block element that's laid out horizontally like an inline element



Flow Layout

- In flow layout you can define the width of (inline-)block elements…
  - …using an absolute or relative unit (px, %,…)
  - …using keywords (min-content, max-content,…)
- Additionally, there are min-width/max-width properties available.
- You can do similar things with height, but be careful when setting fixed height or max-height values.
- Usually we want block elements to contain all its content and not limit the height.
- More in: Exploring the complexities of width and height in CSS

Positioned Layout

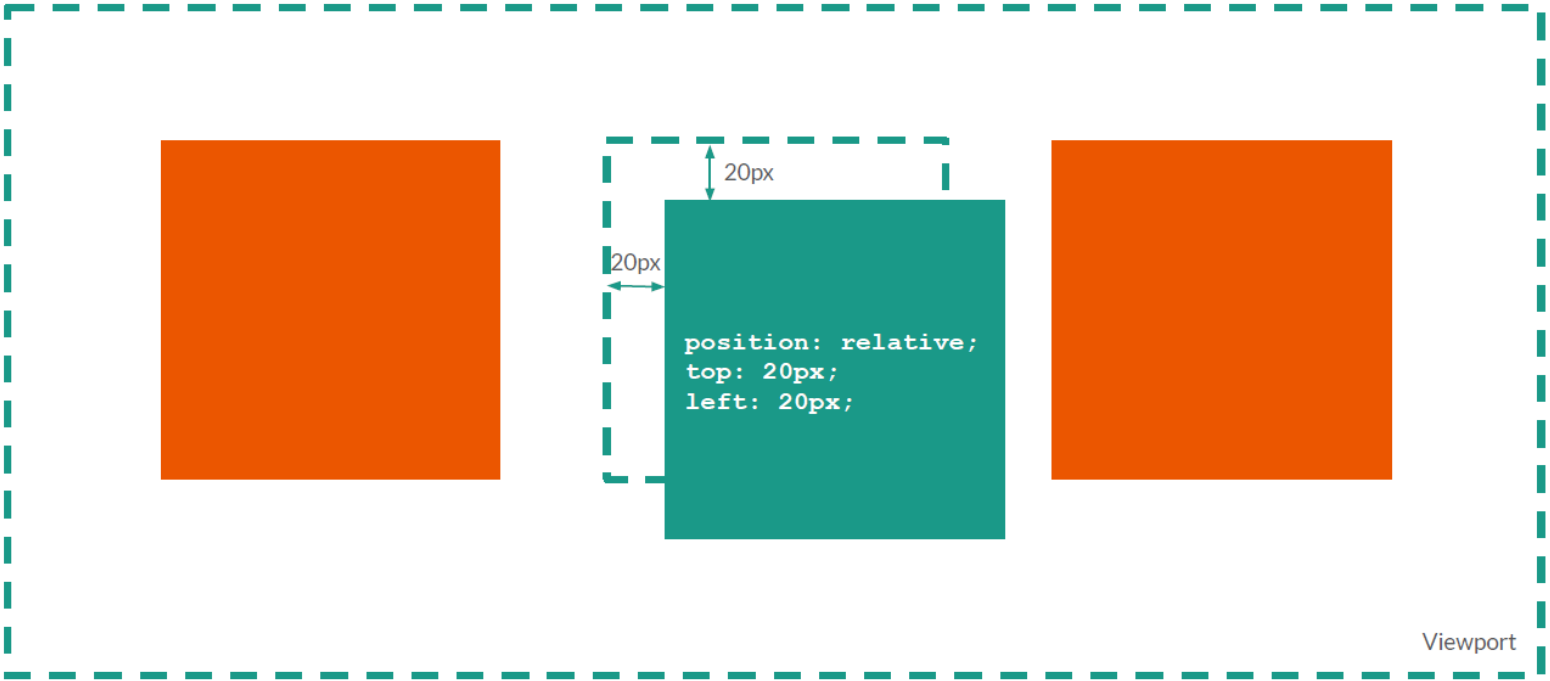


Positioned Layout

- By default the position of all elements on a page is static.
  - … every element is part of the default flow layout.
  - One element after the other; next to each other or below each other.
  - Every element has its own place and pushes the other elements away when it grows.
- When we set “position” to another value than “static” we switch to positioned layout.
  - The available position values are: relative, absolute, fixed, and sticky.
- When we use positioned layout we can use CSS properties to define the element’s position:
  - The properties are: top, bottom, right, and left.
- In positioned layout, elements can overlap.

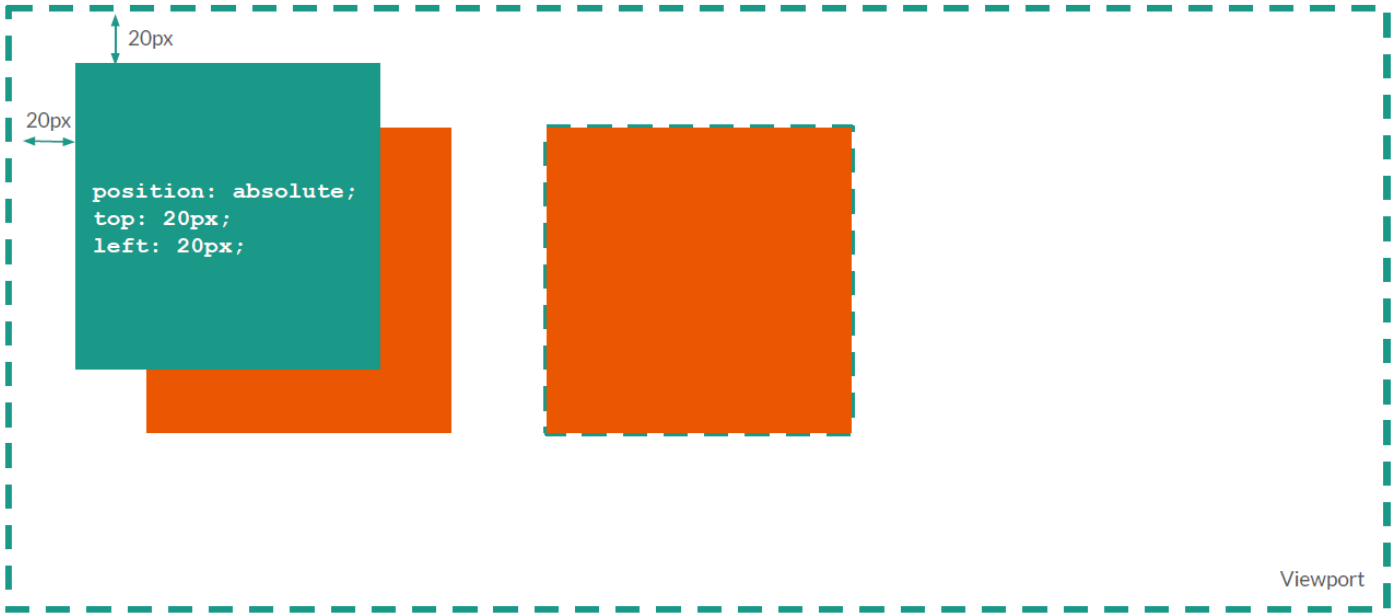
Position relative

- Relative to it’s original position



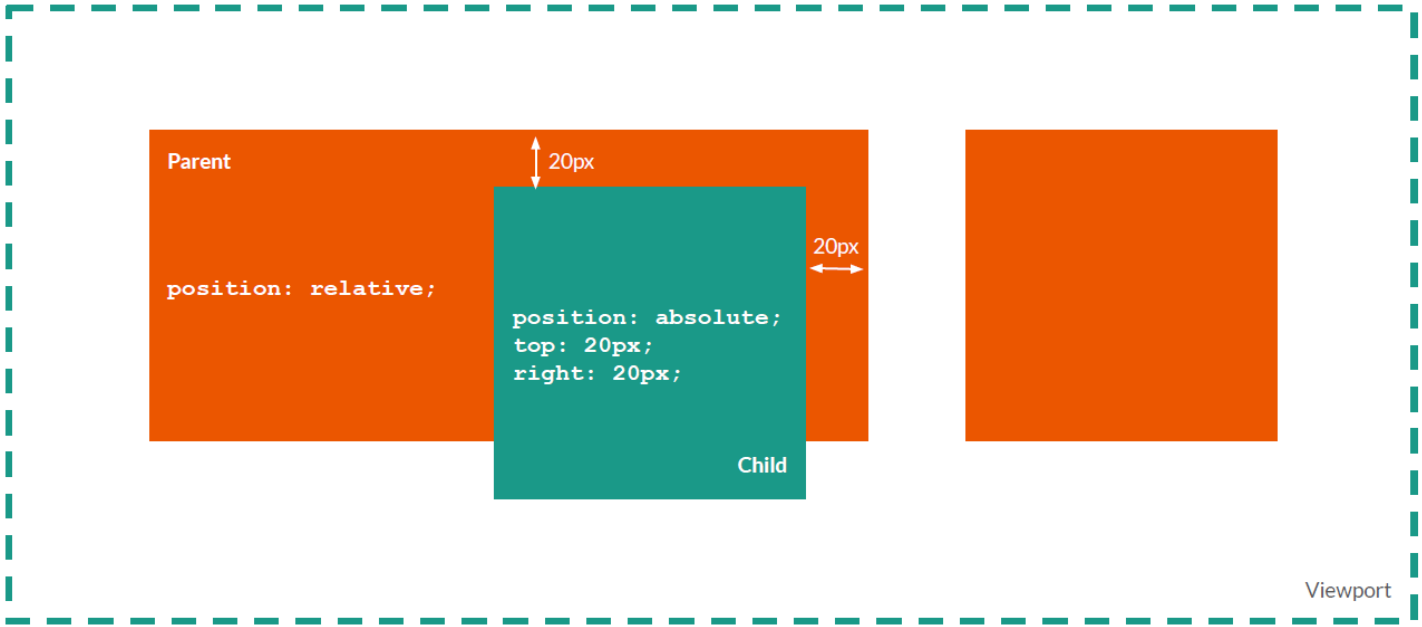
Position absolute

- If parent has position attribute static absolute refers to body element



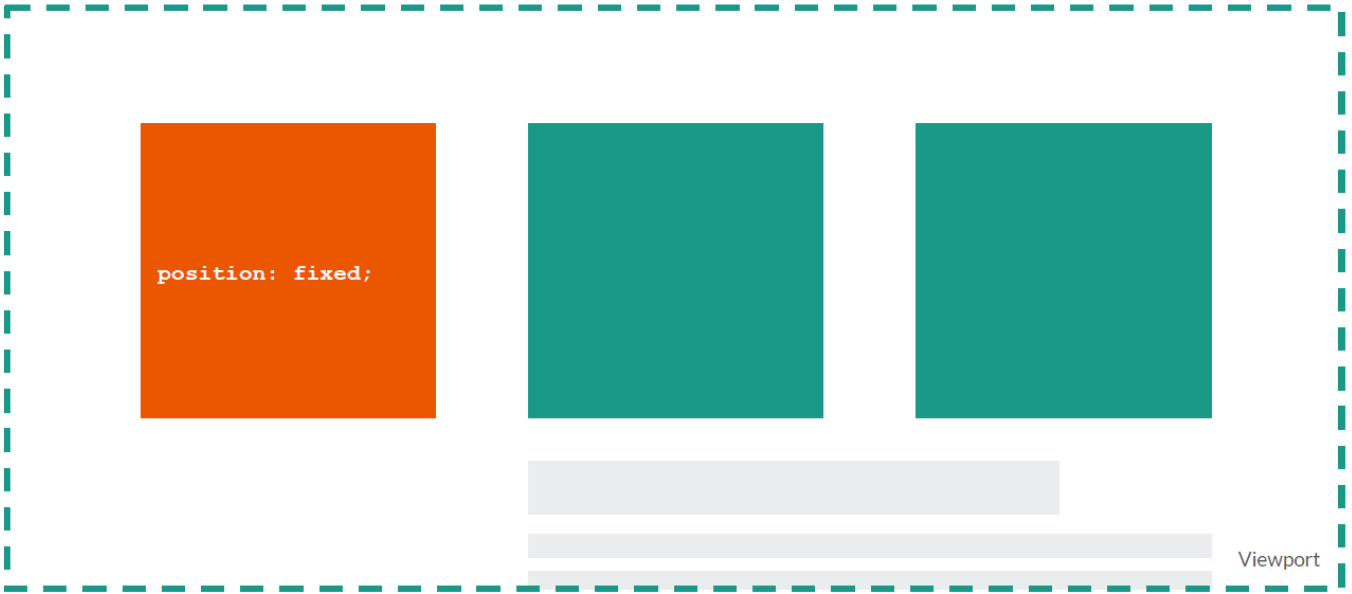
Position absolute

- If parent has position attribute relative absolute refers to the parent element



Position fixed

- Fixed position relative to viewport
- Remains on some position even on scroll



Position fixed



Sticky Position

- Relative until it is in viewport than it acts as absolute

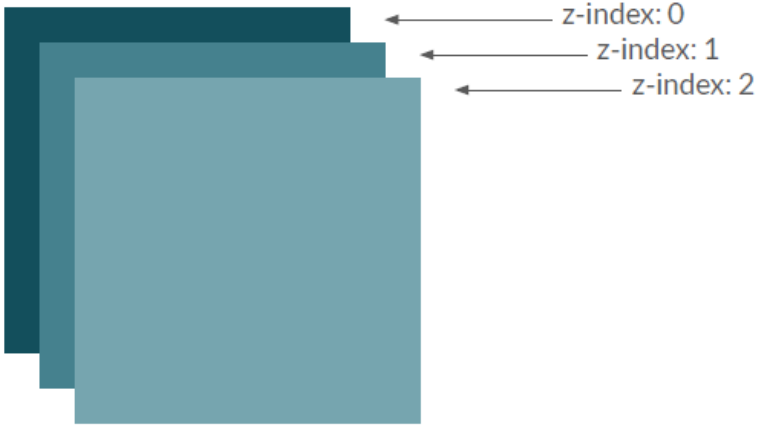


Sticky Position

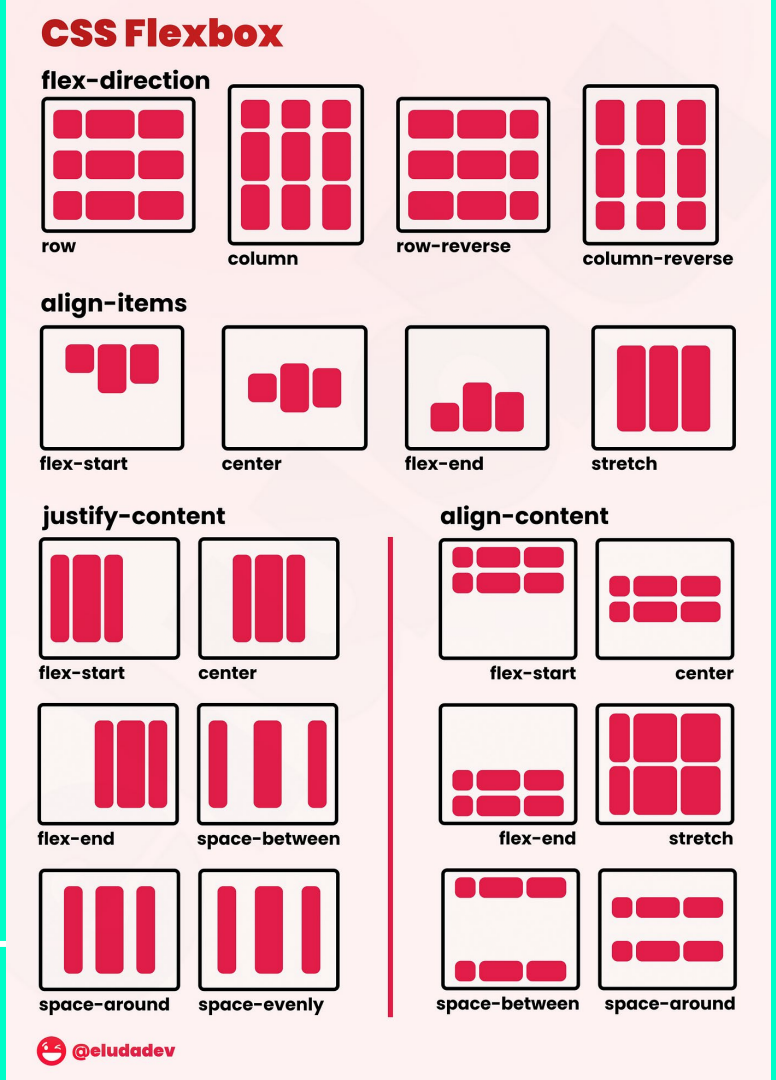


Z-Index

- When elements overlap, the browser needs to decide which element to render on top of the other.
  - Usually, the element that comes later in the DOM is painted on top.
- This behavior can be changed by using the z-index property.
- Works with positioned elements as well as flex and grid items (but not in flow layout).



Flexbox

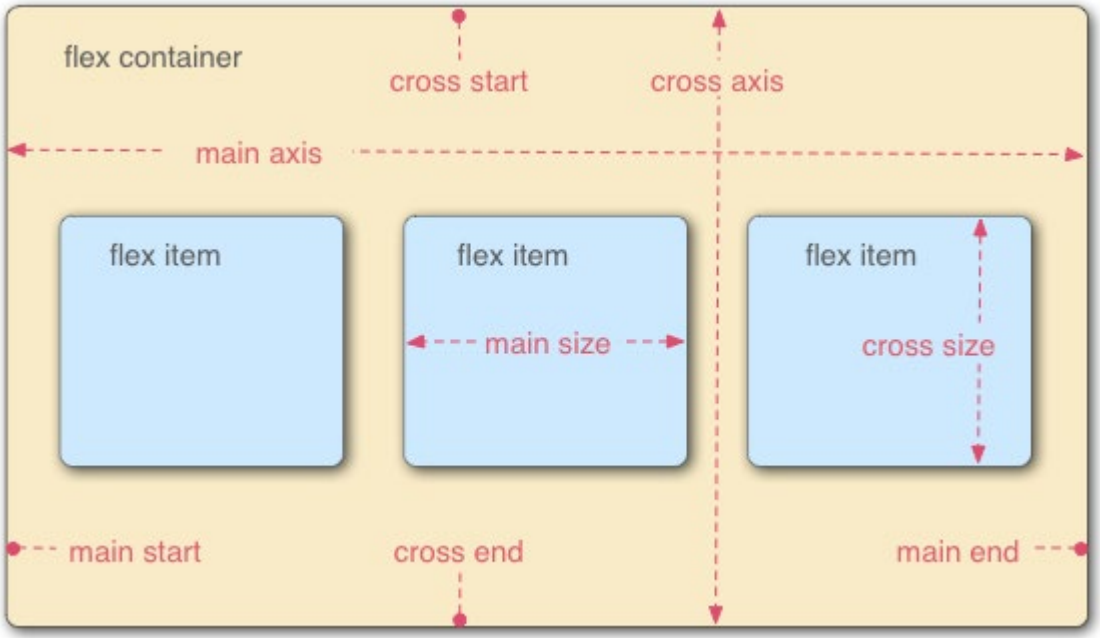

CSS Flexbox
flex-direction
row
column
row-reverse
column-reverse
align-items
flex-start
center
flex-end
stretch
justify-content
flex-start
center
flex-end
space-between
space-around
space-evenly
align-content
flex-start
center
flex-end
stretch
space-between
space-around
@eludadev

Flexbox Basics

- A layout method to arrange elements in one dimension (horizontal or vertical)
- A layout method to perfectly fill the available space in a row.
- Flexbox enables a simple, flexible layout that adapts to the conditions (size of the browser window).
- Flexbox eases the creation of responsive layouts.
- The use of Flexbox also largely eliminates the need for (absolute or relative) size specifications.
- Flex container & flex items, the (direct) children of the container
  - The flex container defines how the flex items are positioned.
  - Flex items can „ignore“ the rules set by the parent and can be manipulated individually.
- Flex items can again become flex containers.
- Guides
  - https://css-tricks.com/snippets/css/a-guide-to-flexbox/
  - https://tympanus.net/codrops/css_reference/flexbox/

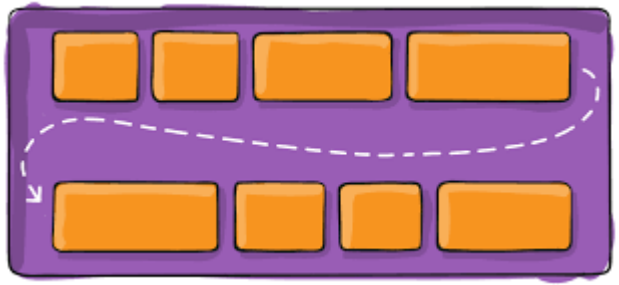
Flexbox Basics

```
.container {
  display: flex;
}
```



- **flex-direction** defines main axis
  - **row** (default): left to right in ltr; right to left in rtl
  - **row-reverse**: right to left in ltr; left to right in rtl
  - **column**: same as row but top to bottom
  - **column-reverse**: same as row-reverse but bottom to top

Flexbox – Container Properties

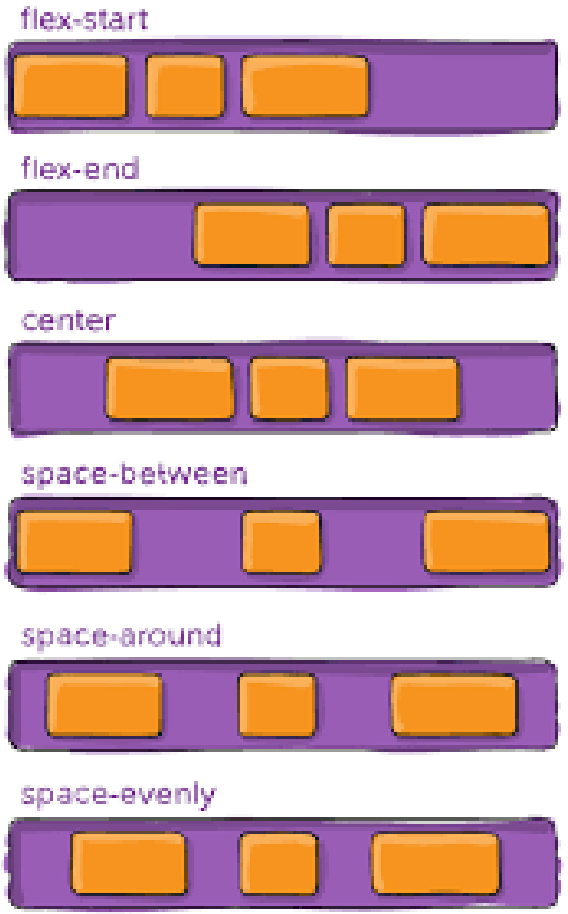


- **flex-wrap**: whether flex items should stay in one line
  - **nowrap** (default): all flex items will be on one line
  - **wrap**: flex items will wrap onto multiple lines, from top to bottom.
  - **wrap-reverse**: flex items will wrap onto multiple lines from bottom to top
- **flex-flow:** shorthand for flex-direction & flex-wrap
  - E.g., column wrap

Flexbox Container Properties

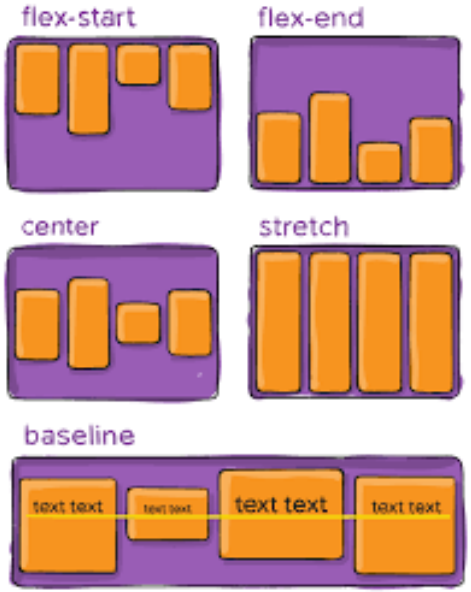
- **justify-content:** alignment along **main axis**
  - **Be aware in case of reverse start & end change!**

  - **flex-start (default):** items are packed toward the start of the flex-direction.
  - **flex-end:** items are packed toward the end of the flex-direction.
  - **center**: items are centered along the line
  - **space-between:** items are evenly distributed in the line; first item is on the start line, last item on the end line
  - **space-around:** items are evenly distributed in the line with equal space around them.
  - **space-evenly:** items are distributed so that the spacing between any two items (and the space to the edges) is equal.



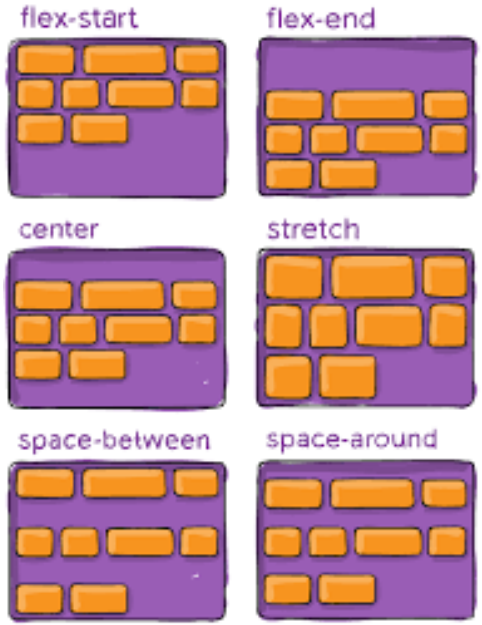
Flexbox Container Properties

- **align-items:** alignment along cross axis
  - **stretch (default):** stretch to fill the container
  - **flex-start:** items are placed at the start of the cross axis
  - **flex-end:** items are placed at the end of the cross axis.
  - **center:** items are centered in the cross-axis
  - **baseline:** items are aligned such as their baselines align



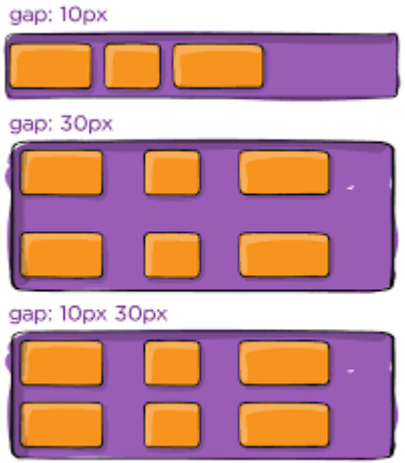
Flexbox Container Property

- **align-content:** alignment of flex container's lines
  - has no effect if there is only one line of content
  - **flex-start**: items packed to the start of the container
  - **flex-end**: items packed to the end of the container
  - **center**: items centered in the container
  - **stretch**: lines stretch to take up the remaining space
  - **space-between**: items evenly distributed; the first line is at the start of the container while the last one is at the end
  - **space-around**: items evenly distributed with equal space around each line
  - **space-evenly**: items are evenly distributed with equal space around them



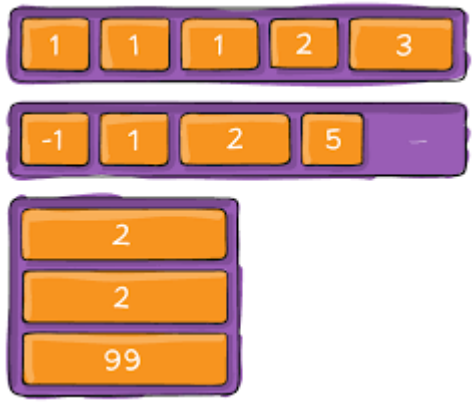
Flexbox Container Property

- **gap, row-gap, column-gap:** explicitly controls the space between flex items.
  - applies spacing only between items not on the outer edges



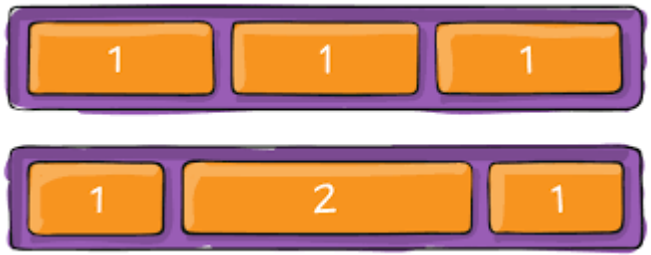
Flexbox Item Property

- **order**: By default, flex items are laid out in the source order. However, the order property controls the order in which they appear in the flex container.
- The initial order of all flex items is **zero** (0)
- Ordering starts with the lowest number (negative values are allowed)



Flexbox Item Property

- **flex-grow:** ability for a flex item to grow if necessary
  - A flex grow factor is a <number> which determines how much the flex item will grow relative to the rest of the flex items
  - If all items have flex-grow set to 1, the remaining space in the container will be distributed equally to all children.
  - If one of the children has a value of 2, that child would take up twice as much of the space as either one of the others (or it will try, at least).

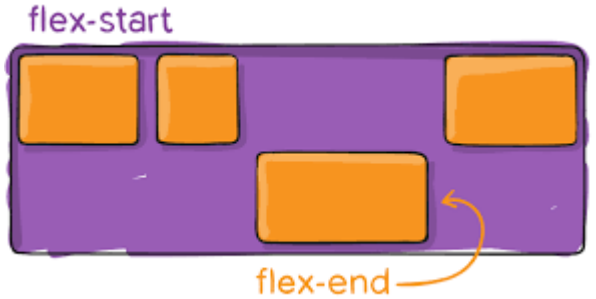


Flexbox Item Property

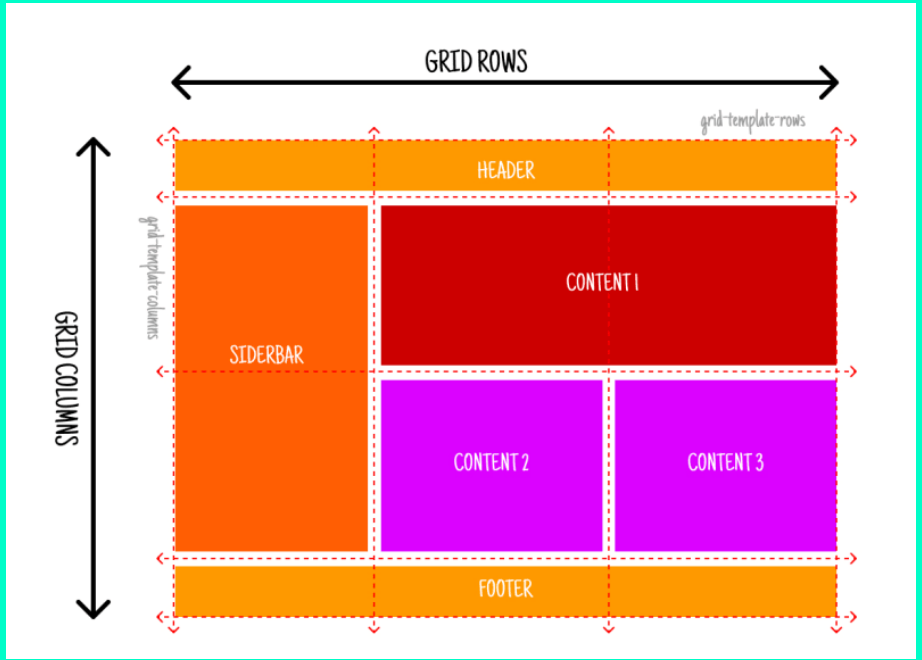
- **flex-shrink**: sets the flex shrink factor of a flex item
  - A flex shrink factor is a <number> which determines how much the flex item will shrink relative to the rest
  - “Opposite” of flex-grow
- **flex-basis**: This defines the default size of an element before the remaining space is distributed.
  - It can be a length (e.g. 20%, 5rem, etc.) or a keyword.
  - The auto keyword means “look at my width or height property”
- **flex**: shorthand for flex-grow, flex-shrink and flex-basis
  - The second and third parameters (flex-shrink and flex-basis) are optional
  - The default is 0 1 auto, but if you set it with a single number value, like flex: 5;, that changes the flex-basis to 0% (flex-grow: 5; flex-shrink: 1; flex-basis: 0%;)
  - It is **recommended** that you use this shorthand property.

Flexbox Item Property

- **align-self:** This allows the default alignment (or the one specified by align-items) to be overridden for individual flex items.
  - Values equal to align-items
  - **center:** Alignment at the middle of the container
  - **flex-start:** Alignment at the beginning of the container (top)
  - **flex-end:** Alignment at the end of the container (bottom)
  - **stretch**: fills the container
  - **baseline**: Alignment on baseline



CSS Grids


GRID ROWS
grid-template-rows
HEADER
grid-template-columns
CONTENT 1
SIDERBAR
GRID COLUMNS
CONTENT 2
CONTENT 3
FOOTER

CSS Grid Layout

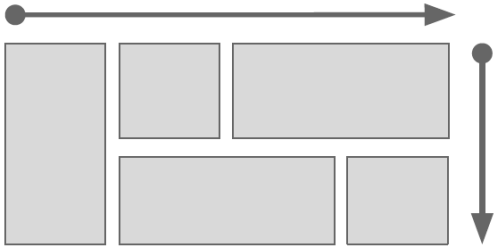
- A layout method to arrange elements in two dimensions.
- A layout method for complex page-layouts.
- Grid container & grid items, the (direct) children of the container.
  - The container defines the rows and columns, areas and item alignment.
  - The item can be placed precisely within the grid and override the alignment specified by the container.
- Grid items can again become grid containers.
- Playground: https://cssgridgarden.com/#de

```
.container {
  display: grid;
}
```

Grid vs Flexbox

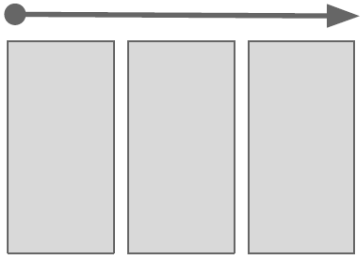
Grid

- Two dimensions(Rows and Columns)
- complete layout with header area, two-column body and a footer
- Layout-first

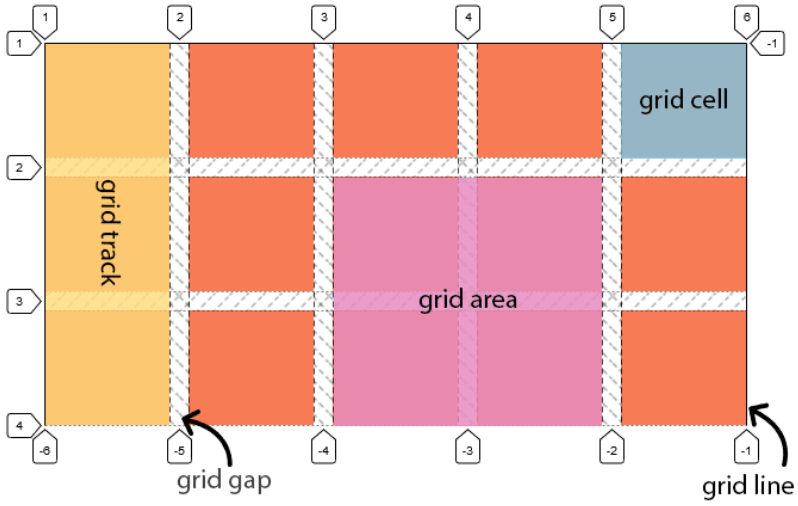


Flexbox

- One dimension
- for stringing together elements of a line, e.g., a main menu
- Content-first



Grid Terminology



- **Grid container:** element, on which display:grid is applied
- **Grid item:** direct children of the grid container
- **Grid line:** The dividing lines that make up the structure of the grid. They can be either vertical (“column grid lines”) or horizontal (“row grid lines”) and reside on either side of a row or column.
- **Grid cell:** The space between two adjacent row and two adjacent column grid lines.
- **Grid track:** The space between two adjacent grid lines. You can think of them as the columns or rows of the grid.
- **Grid area:** The total space surrounded by four grid lines. A grid area may be composed of any number of grid cells.

Properties for Grid Container – Defining Rows and Cols

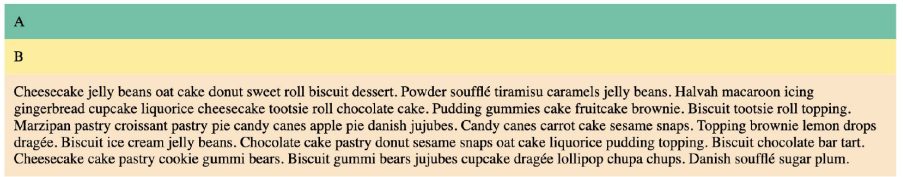
- **grid-template-columns & grid-template-rows**
  - Defines the columns and rows of the grid with a space-separated list of values, i.e., track size.
  - Track size: an be a length, a percentage, or a fraction of the free space in the grid using the fr unit
    - The fr unit allows you to set the size of a track as a fraction of the free space of the grid container.
    - Auto - space is automatically calculated based on size of children
  - repeat-Fct: allows to definition of multiple rows/cols
    - `grid-template-rows: repeat(4, 1fr);`
  - Rows/Cols can also be named
    - `grid-template-columns: [first] 40px [line2] 50px [line3] auto [col4-start] 50px [five] 40px [end];`

Properties for Grid Container – Defining Rows and Cols

```
grid-template-columns: 1fr 1fr 1fr;
grid-template-columns: repeat(3, 1fr);
```




```
grid-template-rows: repeat(3, auto);
```




```
grid-template-columns: repeat(10, 1fr);
```



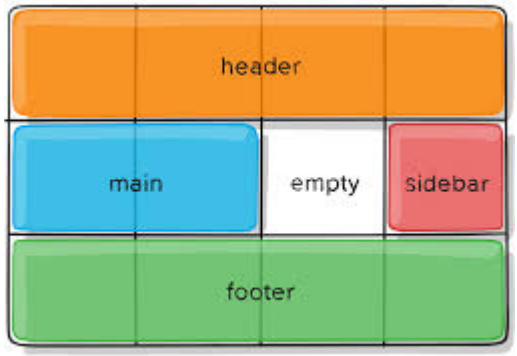
bessere Sichtbarkeit mit Google Chrome Webdevelopement-Tool

Properties for Grid Container – Defining areas

- **grid-template-areas**
  - Defines a grid template by referencing the names of the grid areas which are specified with the grid-area property of the grid item.
  - Repeating the name of a grid area causes the content to span those cells.
  - A period (.) signifies an empty cell.

```
.item-a {
  grid-area: header;
}
.item-b {
  grid-area: main;
}
.item-c {
  grid-area: sidebar;
}
.item-d {
  grid-area: footer;
}
```

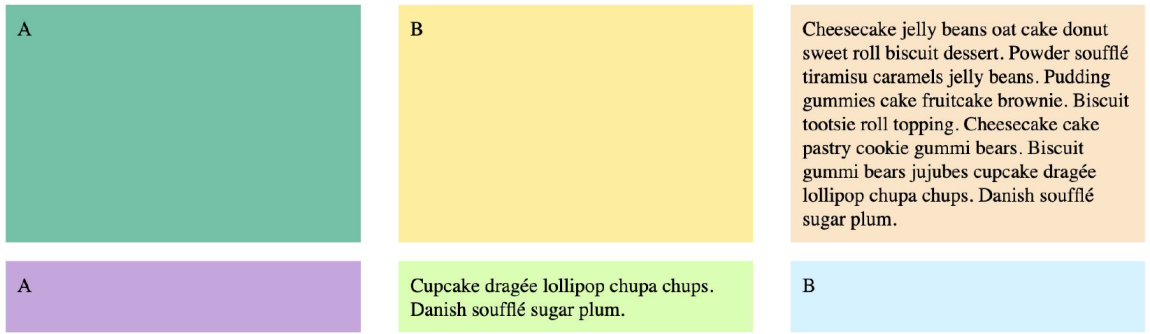
```
.container {
  display: grid;
  grid-template-columns: 50px 50px 50px 50px;
  grid-template-rows: auto;
  grid-template-areas:
    "header header header header"
    "main main . sidebar"
    "footer footer footer footer";
}
```



Properties for Grid Container – Gap

- **column-gap & row-gap**
  - Specifies the size of the grid lines.
  - You can think of it like setting the width of the gutters between the columns/rows.
  - gap can be used as shorthand for both (row-gap column-gap)

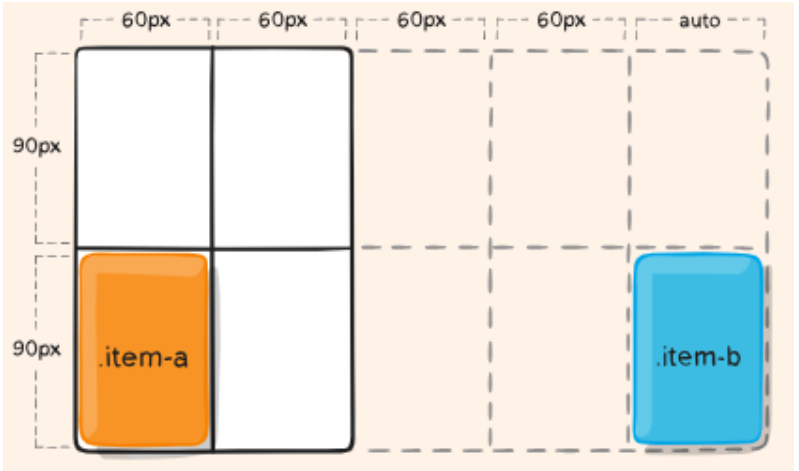
```
grid-template-columns: repeat(3, 1fr);
grid-template-rows: repeat(2, auto);
row-gap: 1em;
column-gap: 2em;
```



Properties for Grid Container – Auto Cols & Rows

- **grid-auto-columns/grid-auto-rows**
  - Specifies the size of any auto-generated grid tracks (aka implicit grid tracks).
  - Implicit tracks get created when there are more grid items than cells in the grid or when a grid item is placed outside of the explicit grid.

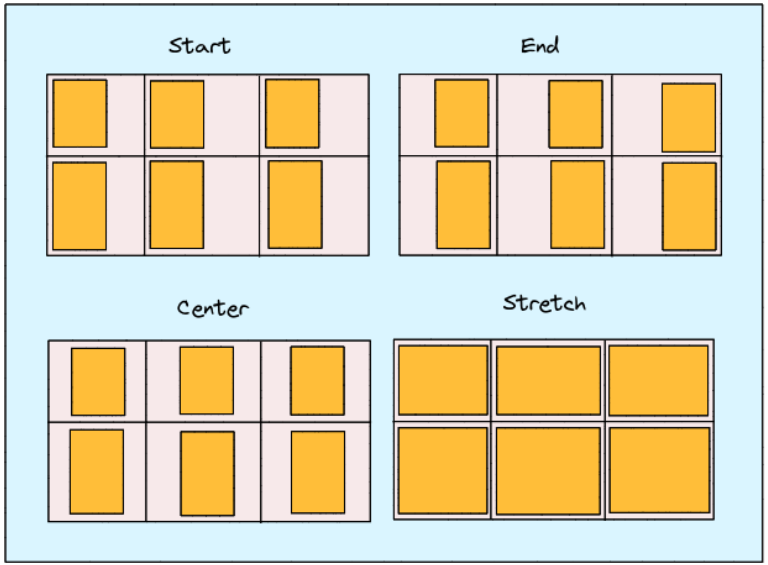
```
.container {
   grid-template-columns: 60px 60px;
   grid-template-rows: 90px 90px;
   grid-auto-columns: 60px;
}
```



Properties for Grid Container

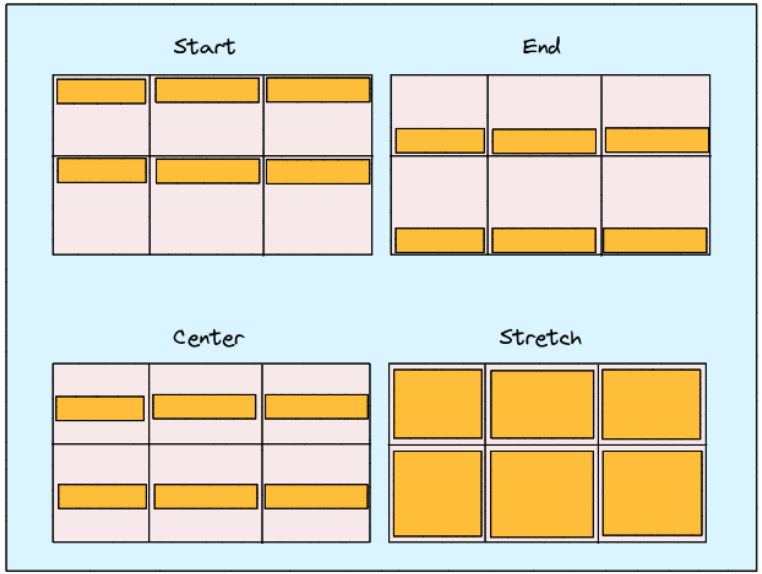
- **justify-items**
  - Alignment of items along row-axis
  - Values: start | end | center | stretch (default)

- **place-items**
  - Shorthand for <align-items> <justify-items>



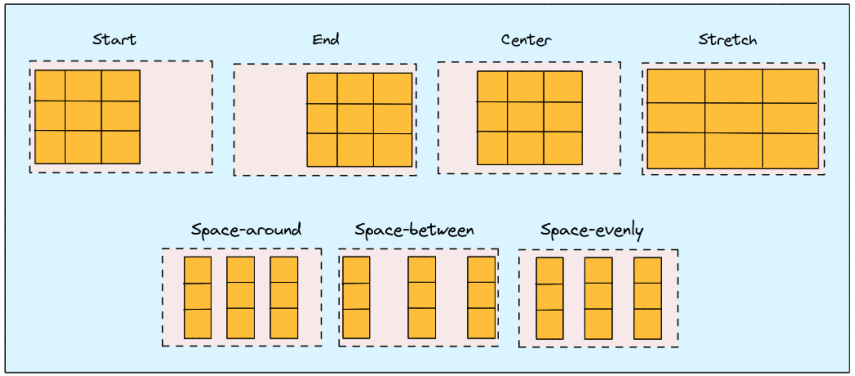
Properties for Grid Container

- **align-items**
  - Alignment of items along column-axis
  - Values: start | end | center | stretch (default)

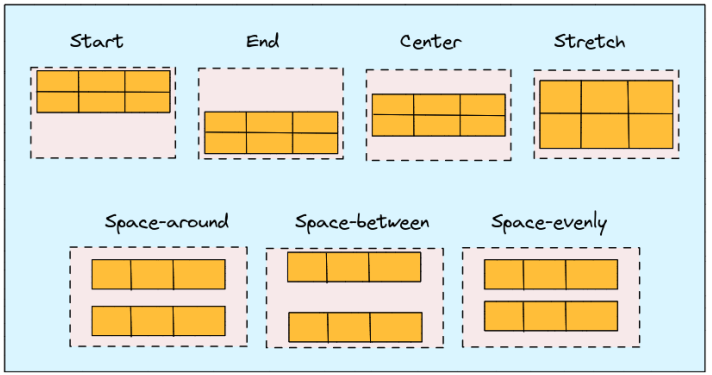


Properties for Grid Container

- **justify-content:** alignment of the grid within its container along the row axis



- **align-content:** alignment of the grid within its container along the column axis



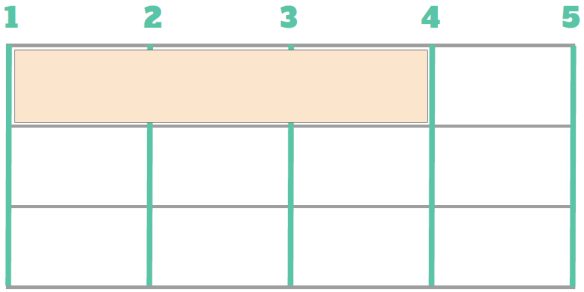
Properties for Grid Items

- **grid-column & grid-row**
  - specify a grid item's location within a grid container

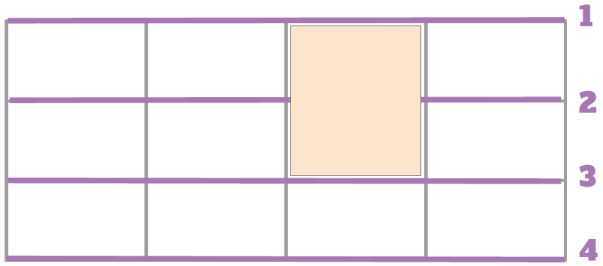
```
.item {
  grid-column: <start-line> / <end- line> |
               <start-line> / span <value>;

  grid-row: <start-line> / <end-line> |
            <start-line> / span <value>;
}
```

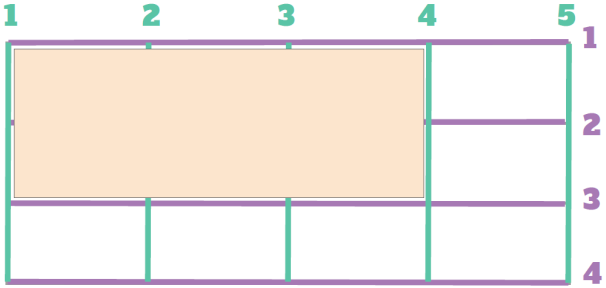
grid-column: 1 / 4;



grid-row: 1 / 3;

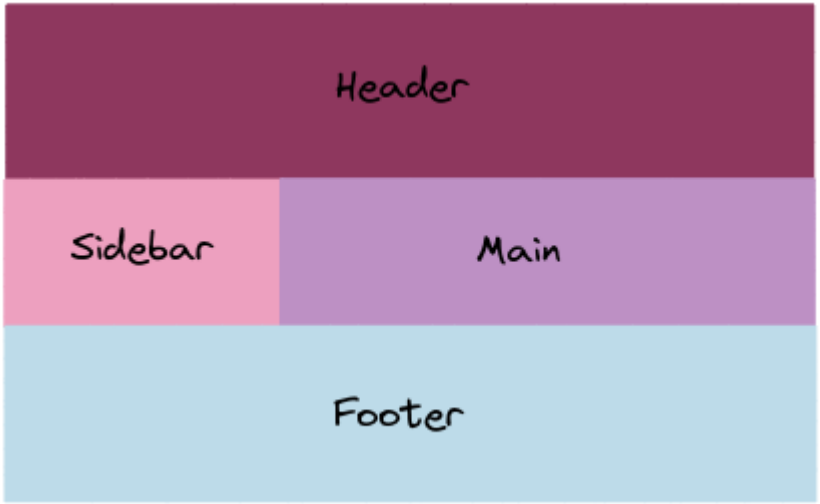


grid-column: 1 / 4;
grid-row: 1 / 3;



Properties for Grid Items

- **grid-area**
  - The grid-area property is used to assign a grid item to a named grid area, specified by the grid-template-areas property on the parent container.




```css
.container {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-rows: 80px 1fr 80px;
  grid-template-columns: 200px 1fr;
  background-color: #DAF5FF;
}

.header {
  grid-area: header;
  background-color: #89375F;
}

.sidebar {
  grid-area: sidebar;
  background-color: #E8A0BF;
}

.main {
  grid-area: main;
  grid-row-start: 2;
  grid-row-end: 4;
  background-color: #BA90C6;
}

.footer {
  grid-area: footer;
  background-color: #C0DBEA;
}
```

Properties for Grid Items

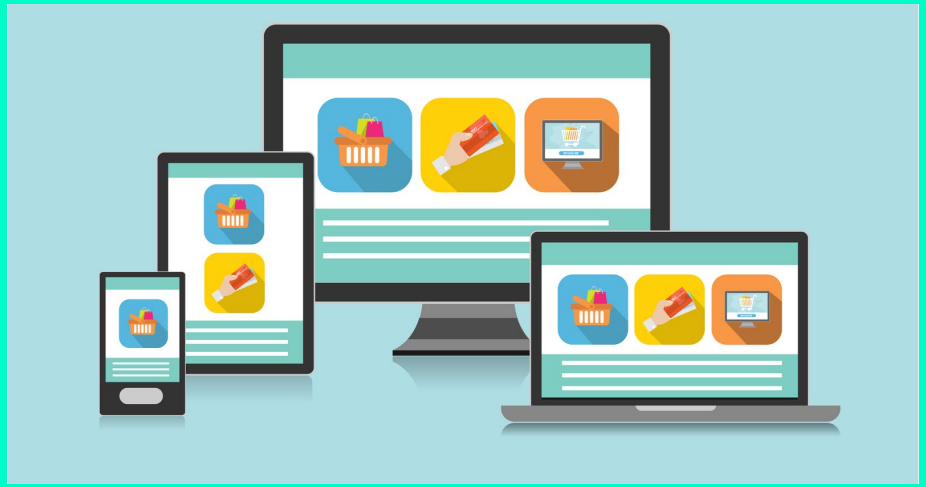
- **justify-self**
  - controls the **horizontal** alignment of a grid item within its cell
  - Values: start | end | center | stretch | auto
- **align-self**
  - controls the **vertical** alignment of a grid item within its cell
  - Values: start | end | center | stretch | auto.
- **place-self**
  - shorthand property
  - Values: <justify-self> <align-self>

Grid: special units, functions and keywords

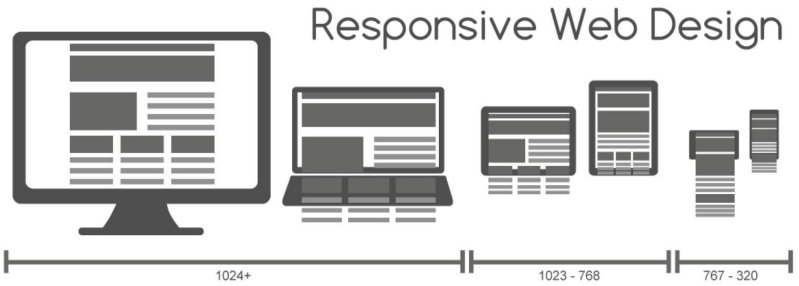
- **fr**: fraction of the free space in the grid (grid-template-columns: 1fr 2fr;)
- **minmax():** to set boundaries for flexible units
  - grid-template-columns: 1fr minmax(300px, 2fr);
  - The second column will never shrink further than 300px.
- **repeat():** repeat parts of your column/row definitions (grid-template-columns: repeat(3, 1fr);)
- **auto-fit & auto-fill:** can be used instead of a number in repeat notation
  - repeat(auto-fit, minmax(200px, 1fr)) or
  - repeat(auto-fill, minmax(200px, 1fr))
  - In case there is space left in the first grid line, auto-fit collapses empty columns to a width of 0.

Responsive Design and Media Queries

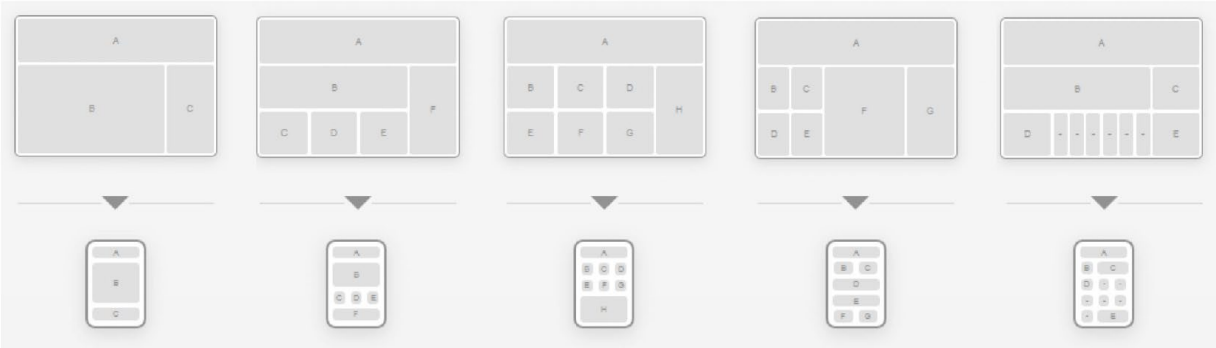
Oh, no… it looks ugly on the smartphone



What is responsive design?



- „Responsive web design“ (RWD) describes an approach to make websites look good on a variety of devices, independent of their screen sizes.
- Mobile First approach
- Fluid and adaptive design (breakpoints)
- Content Choreography



Responsive Design – How?

- **Media queries**
  - CSS module to target specific media types (screen, print) and features (min-width, max-width, resolution, prefers-color-scheme, prefers-reduced-motion,…)
- **Responsive images**
  - Make image resize on smaller screens and only load the image size and file format you need using the <picture> tag and the srcset and sizes attributes
- **Fluid layouts**
  - Use flex and grid layouts with their special features, viewport units and the clamp() function.
  - clamp() CSS function clamps a middle value within a range of values between a defined minimum bound and a maximum bound.
    - function takes three parameters: a minimum value, a preferred value, and a maximum allowed value.
- **Fluid typography**
  - Make the font size adapt to the viewport width using the vw unit and the clamp() function.

```
<meta name="viewport“
      content="width=device-width, initial-scale=1">
```

meta tag gives the browser instructions on how to control the dimensions and scaling of the page

width can be a fixed width e.g. 600px or device-width

CSS Media Queries

- Consist of a media type (screen, print) and one or more features that can be either true or false.
  - Features can be combined by using logical operators.
  - If the media type matches and all features evaluate to true the corresponding styles are applied.

```
@media (max-width: 720px) {
/* tablet and smaller style go here */
}
@media (min-width: 500px) and (max-width:
600px) {
/* layout 500-600px style go here */
}
```

```
@media print {
/* print style go here */
}
@media (orientation: landscape) {
/* landscape style go here */
}
```

CSS Media Queries – Best Practices

- always in one direction (max- or min-width)
- as little as possible in media queries & as much as possible through normal instructions (e.g., images maximum width)
- Media queries are slowly being replaced by new CSS instructions
- Mobile-first (most end devices today are smartphones)

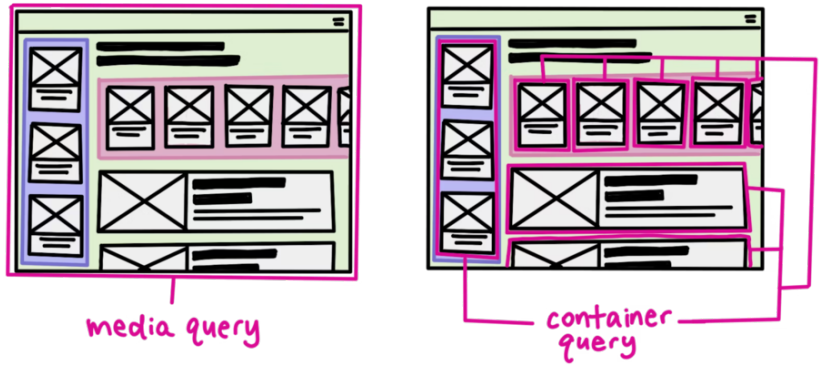
What media queries can't do...

- Imagine you build a component library with many different components like buttons, form fields, cards, media elements, progress bars, and more…
- Another developer, using the components you built, can place the components anywhere. You never know where your components will end up and how much space they will have in a given layout.
- With media queries you can change the look/layout of a component only based on the total screen size, which leads to the following problem…



- The same card component (orange border) is placed in the main section (1) and in the sidebar (2). The viewport width is the same for both views, but the card component styles for this viewport size don’t work well when the card doesn’t have enough space…

Container queries



- Container queries solve this problem
  - New since 2023
- Instead of writing a query and checking the viewport's min- or max-width, we can provide styles based on the item's container size.
  - Two styles of container queries
    - Style queries: checking if a CSS property has a certain value
    - Size queries: ask for the container's size
      - Container needs to set the **container type**
- You can use the same component in different places on your website/app and the layout adjusts perfectly depending on its container's size.

Container Queries

- Container queries require a size container.
  - represents a reference point, or containment context, for container queries
    - For queries in the inline direction, set **container-type: inline-size** (width),
    - for queries in both directions, set **container-type: size;** (width and height)
- The actual container query is similar to a media query and is created by the rule **@container**.

```
<div class="post">
  <div class="card">
    <h2>Card title</h2>
    <p>Card content</p>
  </div>
</div>
```

```
.post {
  container-type: inline-size;
}
```

```
/* Default heading styles*/
.card h2 {
  font-size: 1em;
}
/* If container is larger than 700px */
@container (min-width: 700px) {
  .card h2 {
    font-size: 2em;
  }
}
```

Responsive Images

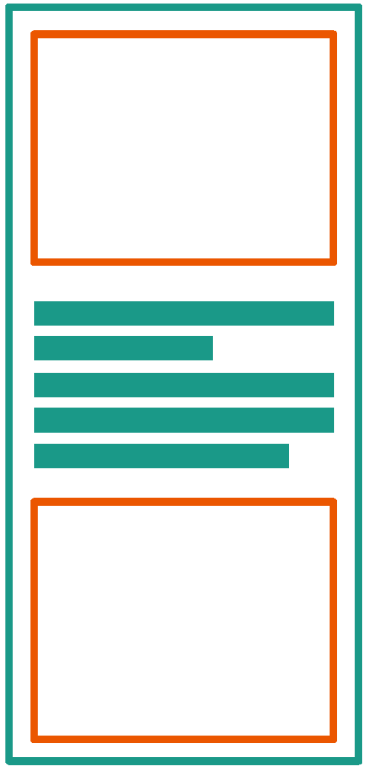
- We don’t want to display a large image on small screens (and load more data than necessary) or have to scale up a small image on large screens.
- We always want to see a perfectly cropped image when there’s not enough space to show the full picture.
  - Use max-width property
- Problem: Browser does not know how large an image is before it loads it
  - large images are important for high resolutions (e.g. Retina displays) - but performance is poor on mobile devices
- srcset and sizes for images
  - give the browser a choice of different sized images so that it can select the appropriate one (either image width or resolution)

Responsive Images

```
<img src="minerva.jpg" alt="A portrait shot of Minerva McGonagall"
srcset="minerva-400.jpg 400w,
minerva-800.jpg 800w,
minerva-1200.jpg 1200w"
sizes="(max-width: 800px) 100vw,
(max-width: 1000px) 80vw,
60vw" />
```

- **srcset** defines the set of images we will allow the browser to choose between, and what size each image is.
- **sizes** defines a set of media conditions (e.g. screen widths) and indicates what image size would be best to choose, when certain media conditions are true

```
sizes="(max-width: 800px) 95vw, 60vw"
```



Small screen:
- images' width: 95vw

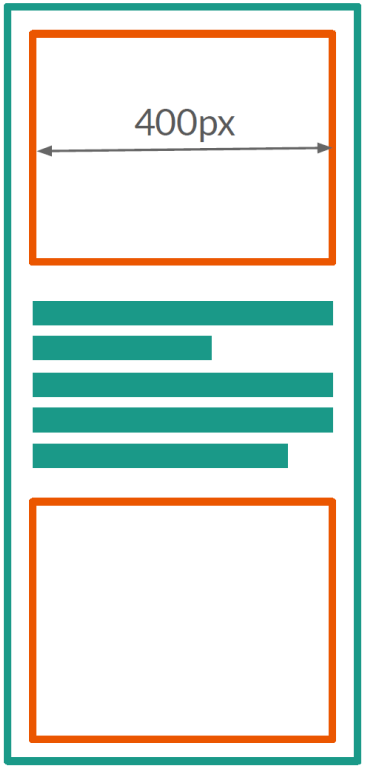


Large screen:
- images' width: 60vw

Responsive Images

```
srcset="minerva-400.jpg 400w,
minerva-800.jpg 800w,
minerva-1200.jpg 1200w"
```

- The browser can choose from 3 images depending on the device's pixel density.
  - device pixel ratio of 1.0: minerva-400.jpg
  - device pixel ratio of 2.0: minerva-800.jpg (because an image twice the size is needed to fill the available device pixels)



Responsive Images

- The **<picture>** element can be used to provide…
  - different versions of the image when it comes to aspect ratio (e.g. a square image to be used on mobile screens but a 16:9 image on large screens)
  - different image file types to provide newer image formats for browser that support them and fallbacks for older browsers

Responsive Images

```
<picture>
  <source media="(max-width: 600px)" srcset="minerva-square.jpg" />
  <source media="(min-width: 601px)" srcset="minerva-wide.jpg" />
  <img src="minerva.jpg" alt="A portrait shot of Minerva McGonagall" />
</picture>
```

- On screens up to 600px width the image minerva-square.jpg is loaded.
- On screens starting from 601px width the image minerva-wide.jpg is loaded.
- Browsers not understanding the picture element use the good old img element and loadminerva.jpg.





Responsive Images

```
<picture>
<source srcset="minerva.webp" type="image/webp" />
<img src="minerva.jpg" alt="A portrait shot of Minerva McGonagall" />
</picture>
```

- Browsers that support the image/webp file format load minerva.webp.
- Other browsers load minerva.jpg.

Useful Things in CSS

CSS Custom Properties

- Custom properties make it possible to reuse property values in a CSS file.
- Declaration: A custom property always starts with a double hyphen (--) and is case-sensitive.
  - `--main-color: #49735f;`
  - `--body-font-family: Helvetica, Arial, sans-serif;`
- Usage: Use the var() function to access the custom property's value.
  - `color: var(--main-color);`
  - `font-family: var(--body-font-family);`

CSS Custom Properties

- Custom properties can be declared in any rule block in a CSS file and are then available for the
- selected element an all its children; they inherit the custom property.

```
article {
  --accent-color: #a66460;
}

.article-header {
  color: var(--accent-color);  /* works */
}

footer {
  color: var(--accent-color);  /* does not work */
}
```

```
<article>
  <header class="article-header">
    <h1>CSS Custom properties</h1>
  </header>
</article>
<footer>
  <p>Some text…</p>
</footer>
```

CSS Custom Properties

- To provide a fallback value in case the CSS custom property used is not available use the following syntax:
  - `var(--accent-color, deeppink);`
- To make values available throughout your CSS it's common to use the :root pseudo-class selector.
  - The :root selector matches the root element in a document tree, which is the <html> element in HTML files.

```
:root {
  --accent-color: #a66460;
  --text-color: #242424;
}
```

Import of Stylesheets

- CSS Files can get rather long -> separate them into several ones
- The **@import** CSS at-rule is used to import style rules from other valid stylesheets.
- An @import rule must be defined **at the top** of the stylesheet, before any other at-rule

```
@import "custom.css";
@import url("landscape.css") screen and (orientation: landscape);
```